



Technische Universität Berlin

Faculty IV - Lab: Computer Vision and Remote Sensing

Computer Science (B.Sc.)

Bachelor Thesis

MULTIMODAL DEEP ENSEMBLES FOR
THE SEMANTIC ANALYSIS OF
EARTH OBSERVATION IMAGERY

Submitted by:

Damian Kunze

kunze.2@campus.tu-berlin.de

11.09.2026

Advisor:

Dr.-Ing. Ronny Hänsch

Reviewers:

Prof. Dr.-Ing. Olaf Hellwich

Prof. Dr. Begüm Demir

Kurzfassung

Die semantische Segmentierung multimodaler Fernerkundungsdaten stützt sich üblicherweise auf eine einzelne, feste Kombination von Eingabekanälen für ein Modell, wodurch die Vielfalt der verfügbaren Modalitäten ungenutzt bleibt. Diese Arbeit begegnet dem, indem Ensembles aus *UNet*-Segmentierungsmodellen trainiert werden, bei denen die Eingabekanäle jedes Ensemble-Mitglieds variiert werden. Dafür werden sechs Kanalmodulationsstrategien vorgeschlagen: *PCA on random subsets*, *Powerset of all modalities*, *Random channel drop*, *Random drop modality percentage*, *Random linear combination* und *Random classes simplified LDA*. Sie werden auf den multimodalen Datensätzen *DFC18* und *MDAS* evaluiert, um zu untersuchen, ob modulierte Ensembles einem einzelnen Baseline-Ensemble überlegen sind. Zu den zentralen Erkenntnissen zählt: Bereits bei unmodulierten Daten verbessert die Ensemble-Strategie die Average Accuracy, die hier als Baseline für die modulierten Strategien dient. Darüber hinaus schneiden die Modulationen *PCA on random subsets* und *Powerset of all modalities* am besten ab, was für beide Datensätze zutrifft. Die Kombination von *PCA on random subsets* mit *Powerset of all modalities* liefert mit 65% Average Accuracy (11% über der Baseline) konkurrenzfähige Ergebnisse im Vergleich zu anderen Ansätzen, was bestätigt, dass das Ziel diverser Ensembles eine effektive, datensatzübergreifende Strategie ist.

Abstract

Semantic segmentation of multimodal remote sensing data typically relies on feeding a single, fixed combination of input channels into one model, leaving the diversity across available modalities unused. This thesis addresses this by training ensembles of UNet segmentation models with varying input channels for each ensemble member. Six channel modulation strategies are proposed for this purpose: *PCA on random subsets*, *Powerset of all modalities*, *Random channel drop*, *Random drop modality percentage*, *Random linear combination* and *Random classes simplified LDA*. They are evaluated on the multimodal Datasets *DFC18* and *MDAS* to investigate whether modulated ensembles outperform a single baseline ensemble. Key findings include: The ensemble strategy already improves Average Accuracy for unmodulated data, which serves here as the baseline for the modulated strategies. Beyond that, the modulations *PCA on random subsets* and *Powerset of all modalities* perform best, which holds for both datasets. The combination of *PCA on random subsets* with *Powerset of all modalities* yields competitive results of 65% Average Accuracy (11% over baseline) compared to other approaches, which confirms that creating diverse ensembles in this way is an effective dataset-general strategy.

Contents

Kurzfassung	i
Abstract	i
Contents	iii
1 Introduction	1
2 Background & Related Work	3
3 Methods	7
3.1 Channel Modulations	7
3.1.1 Identity	8
3.1.2 Random Channel Drop	8
3.1.3 Random Drop Percentage of Modality	9
3.1.4 Powerset of all modalities	10
3.1.5 Principal Component Analysis (PCA)	11
3.1.6 Random Linear Combination	14
3.1.7 Random Classes Simple LDA	14
3.2 Ensembles	16
3.2.1 Prediction	16
3.2.2 Testing	16
3.3 Ensemble Members	17
3.3.1 Training	18
3.3.2 Class weights	19
4 Experiments	21
4.1 Datasets	21
4.1.1 Dataset of the 2018 IEEE GRSS Data Fusion Contest (DFC2018)	21
4.1.2 MDAS Dataset	23
4.2 Performance Metrics	26
4.2.1 Accuracies	27
4.2.2 Cohen's Kappa	27
4.2.3 Multiclass DICE Score	28
4.3 Training the Ensembles	28
4.4 Results on DFC18	30
4.4.1 Baseline	30
4.4.2 Using Modulated Ensembles	31
4.4.3 PCA	32
4.4.4 Powerset of all modalities (chained with PCA)	34
4.4.5 Random channel drop	35
4.4.6 Random Classes Simple LDA	37
4.5 Results on MDAS	39
4.5.1 Baseline	39
4.5.2 Using Modulated Ensembles	39
4.5.3 Powerset of all modalities	40
4.6 Conclusions	43
Bibliography	45

Introduction

Urban land-use and land-cover monitoring is an important application of multimodal Earth observation imagery. **Earth observation imagery** means remote sensing image data depicting the Earth, usually collected by satellites or research planes. The task addressed on this data is **semantic segmentation**, also referred to as semantic analysis: Labeling image data pixel-wise with predefined classes. **Multimodal** data refers to data gathered from multiple sensors covering the same object, rather than from a single sensor such as an RGB camera alone.

A well-established approach for improving prediction quality on such tasks is **Ensemble Learning (EL)**: Learning with multiple models, referred to as “ensemble members” or “base learners”, each of which produces its prediction independently, with the ensemble returning an aggregation as the final result. A model or ensemble is considered **strong** if its testing accuracy is comparatively high and the members of an ensemble are considered **diverse** if the errors they make are mostly uncorrelated, since averaging predictions whose errors are largely uncorrelated tends to cancel these errors out in the final ensemble prediction. Combining strong and diverse ensemble members is well known to outperform relying on a single strong model alone. The central challenge for any concrete ensemble design is how to actually construct members that are both strong and diverse.

For multimodal data, the input channels available to a model offer a natural lever for introducing diversity between ensemble members, for example by giving each member access to a different subset or combination of channels.

This idea for multimodal semantic segmentation is investigated in this thesis, building on **Deep Ensembles**: Deep neural networks, namely **Convolutional Neural Networks (CNNs)**, with the **UNet** neural network architecture used as the ensemble’s base learners, combined with an ensemble **bagging** method, meaning that base learners are each trained on different data (or a subset thereof). The goal is to test whether varying the input channels seen by each ensemble member, through a proposed set of **Channel Modulation** strategies, succeeds in building ensemble members that are stronger and/or more diverse and whether this in turn improves segmentation accuracy over a single strong baseline trained on the full unmodulated input.

This thesis makes the following contributions:

1. A unified matrix-based Channel Modulation formalism ($\mathbf{M}^{C \times K}$) that expresses both hard selection (*Powerset of all modalities*, *Random channel drop*, *Random drop modality percentage*) and projection (*PCA on random subsets*, *Random linear combination* and *Random classes simplified LDA*) as instances of the same operation.
2. An empirical comparison of these six modulation strategies for generating ensemble diversity, benchmarked against an unmodulated (*Identity* modulation) baseline.
3. Cross-dataset validation on the two multimodal datasets: *DFC18* Section 4.1.1 and *MDAS* [1].

A few further notational conventions are used throughout this work: The **indicator function** $\mathbf{1}\{\dots\}$ evaluates to 1 for True values and 0 for False values. Matrices $\mathbf{X}$ are indexed using parenthesis notation, $\mathbf{X}(c, k) = x_{c,k}$, denoting the value in the c -th row and k -th column. **Marginal frequencies** refer to how often a class relatively appears in a (test-) set.

The code of this thesis is available on Github at:

https://github.com/damnkunze/Thesis_Multimodal-Deep-Ensembles_Semantic-Analysis_Earth-Observation-Imagery

Background & Related Work

Providing different subsets of the dataset to a collection of models is known as **Ensemble Learning with bagging**: A collection of different models is created by varying their input data and a “summarized” prediction, the ensemble prediction is then formed in bagging’s aggregation step, as described in “Bagging Predictors” [2]. Rather than the paper’s primary majority voting over hard class labels, here each member’s class **probabilities** are **averaged** before the final class is taken, an alternative Breiman also discusses and finds to perform comparably.

The concept of **ensemble diversity** has long been established, as stated by Krogh and Vedelsby: “A combination of the output of several networks (or other predictors) is only useful if they disagree on some inputs” [3].

A strong model architecture for semantic image segmentation is the **UNet** [4]. Originally developed to label microscopy imagery, it has also worked well on Earth observation imagery and is described in more detail in Section 3.3.

Data fusion in remote sensing means combining multimodal data into a single joint representation for a model to train on. It differs on when this fusion takes place: **early** fusion concatenates the raw modalities as input channels before a single model, **intermediate** fusion extracts features per modality separately and merges them partway through the network and **late** fusion classifies each modality separately and combines only the final predictions.

Considerable work has been done on algorithms for semantic segmentation in remote sensing.

The outcome paper “*Advanced Multi-Sensor Optical Remote Sensing for Urban Land Use and Land Cover Classification*” [5] introduces the DFC18 dataset and reports the results of the challenge’s two best-performing teams. The DFC18 challenge was semantic segmentation, with its data fusion track being particularly interesting, since it involved multimodal data acquired at the same time. The winning approaches rely on early or intermediate fusion using class-specific **post-processing** to strongly improve accuracy on difficult classes and reach a best AA of 71.66%. The winning team’s raw model results shown in Figure 2.1 reached only 56.5% AA. LiDAR alone matches the full fusion model both before and after

post-processing, so a single strong modality can be better than full fusion on this dataset, a pattern the results in Table 4.2 also show.

Hong et al. in “*More Diverse Means Better: Multimodal Deep Learning Meets Remote-Sensing Imagery Classification*” [6] compare five fusion architectures within a single model: early, middle, late, encoder-decoder and their novel **cross fusion**. Early fusion here is effectively this thesis’s **Identity** modulation baseline. Despite the title, their notion of diversity is **not ensemble diversity**: They use it to describe both how thoroughly a single model blends modalities internally through its node connections and the benefit of having more information sources available through multimodality. Their central result is that multimodal input beats single-modality baselines by a wide margin.

Building on the framework of Random Forests are **Rotation Forests** [7], which pursue a similar goal of diverse individual members: Each member uses a different random subset of the dataset, on which a **Principal Component Analysis (PCA)** is performed. This inspired the idea of applying PCA to subsets (Section 3.1.5) to generate ensemble members here as well.

Hänsch and Hellwich develop an advanced technique based on **Random Forests (RF)** in [8] to semantically label Earth observation imagery from the DFC18 dataset. Their approach uses multiple instances of an RF classifier, each given a different input modality or combination of modalities from the dataset. The accuracies shown in Figure 2.1 are the results of this approach, compared against **baseline** accuracies on the right, which are NN and/or CNN results **without heavy pre- or postprocessing**, taken from the DFC18 results paper [5].

Keeping modalities separated in the ensembles, without data fusion is explored by Burenko and Vetrov in “*Beyond Modality Fusion: Deep Ensembles for Multimodal Classification*” [9], published very recently in 2026. Each ensemble member is trained on only a **single modality** and they study how many models to allocate per modality. Their central claim is that multimodal data can be classified effectively this way without any explicit fusion, outperforming other approaches tested by them **at equal parameter count**. Their setting differs from this thesis in performing basic classification, not segmentation.

In “*AutoPrognosis-M: Automated Ensemble Multimodal Machine Learning for Healthcare*” [10] ensemble members differ by base model and by **fusion strategy** not by input modalities. The authors find that diversity across fusion strategies beats the single best fusion strategy.

TABLE I
RESULTS OBTAINED BY RFs TRAINED WITH 20k SAMPLES ON DIFFERENT DATA SOURCES (FIRST SIX COLUMNS) AS WELL AS RESULTS OBTAINED BY (DEEP) NEURAL NETWORKS TAKEN FROM [1] (LAST FOUR COLUMNS). BEST DATA-SOURCE DEPENDENT RF-RESULT IS DENOTED IN BOLD FONT

#	class		RGB	HSI	L-Int	L-DSM	LiDAR	All	1 st	2 nd -NN	2 nd -CNN	2 nd -CNN+NN
1	Healthy grass		95.65	98.18	95.15	21.85	94.97	97.48	88.7	99.2	99.4	99.7
2	Stressed grass		56.81	63.24	45.03	20.29	44.60	47.75	87.3	0.0	0.0	0.0
3	Artificial turf		38.06	4.77	16.96	13.49	34.11	64.94	64.14	21.3	94.6	94.6
4	Evergreen trees		97.58	71.55	94.26	70.26	93.89	97.41	97.05	95.6	97.9	98.1
5	Deciduous trees		56.82	36.22	82.39	34.44	83.17	84.31	73.02	59.7	82.9	83.0
6	Bare earth		53.78	33.80	41.47	2.89	55.07	48.05	27.64	5.4	44.0	45.4
7	Water		9.30	3.24	7.22	30.55	33.94	12.52	9.15	96.7	77.5	84.5
8	Residential buildings		63.46	31.58	72.27	82.23	79.60	76.33	75.03	0.0	0.0	0.0
9	Non-residential buildings		42.28	46.22	44.59	73.39	84.36	86.46	93.55	95.1	92.0	93.6
10	Roads		50.56	39.90	51.64	42.54	60.93	57.56	62.44	76.0	53.0	59.9
11	Sidewalks		50.74	44.14	62.35	55.41	68.23	70.17	68.52	69.5	75.4	66.0
12	Crosswalks		38.17	22.46	64.94	22.02	69.04	66.12	7.46	0.0	0.0	0.0
13	Major thoroughfares		28.25	23.84	30.14	22.94	29.07	32.34	59.94	33.3	41.9	30.7
14	Highways		20.89	14.04	19.35	60.59	32.57	34.46	17.95	30.3	25.9	30.3
15	Railways		74.05	6.21	56.76	26.33	51.41	11.99	80.46	85.4	7.2	77.6
16	Paved parking lots		59.56	38.99	80.85	81.55	89.83	85.99	60.8	58.4	69.4	72.3
17	Unpaved parking lots		0.00	0.00	0.00	0.40	0.00	0.00	0	0.0	0.0	0.0
18	Cars		73.94	31.96	75.96	75.22	77.34	76.65	64.33	0.0	0.0	0.0
19	Trains		79.31	50.02	79.11	53.99	92.17	92.78	50.94	89.6	94.1	89.6
20	Stadium seats		29.79	29.82	93.19	21.23	84.08	47.82	41.97	85.1	50.9	86.9
	OA (%)		55.60	39.10	60.55	44.22	67.54	65.90	63.28	51.2	54.5	58.6
	AA (%)		50.95	34.51	55.68	40.58	62.92	59.56	56.5	50.0	50.3	55.6
	κ		0.53	0.35	0.58	0.41	0.66	0.64	0.61	0.48	0.52	0.56

Figure 2.1: Table taken from [8], showing the results of their approach, using [5] as a baseline seen in the columns on the right.

Methods

3.1 Channel Modulations

Let C denote the total number of channels across all modalities of a dataset, H, W the spatial dimensions of the datasets image tiles and B the batch size.

A **Channel Modulation** is a matrix $\mathbf{M}^{C \times K}$ mapping C input channels to K output channels, where each column defines a weighted combination of inputs.

Given a batched input tensor $\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W}$, the modulation is applied channel-wise with simple matrix multiplication of

$$\mathbf{Y}(b, k, h, w) = \sum_{c=1}^C \mathbf{M}(c, k) \mathbf{X}(b, c, h, w) \quad (3.1)$$

so that $\mathbf{Y} \in \mathbb{R}^{B \times K \times H \times W}$. Each output channel k is a linear combination of all input channels at that pixel, with weights given by column k of $\mathbf{M}$. This single formulation covers both hard and soft modulations: A binary $\mathbf{M}$ with exactly one nonzero entry per column acts as a selection matrix, dropping or reordering channels, while a continuous $\mathbf{M}$ computes weighted combinations, e.g. a PCA projection or a random mixing of channels.

The following are **Channel Modulation Functions** f . Most of them take a parameter N_E defining how many modulations to create and each function f returns a list of channel modulations: $f(m) = [\mathbf{M}^{(1)}, \dots, \mathbf{M}^{(N_E)}]$.

From these N_E channel modulations, models 1, ..., N_E are created, each as described in Section 3.3.

The **modulation graphs**, such as Figure 3.1, visualize $\mathbf{M}$: Input channels on the y-axis, grouped by modality and output channels on the x-axis. Each cell (c, k) is colored by $\mathbf{M}(c, k)$: blue for negative, red for positive, white for near-zero weights. A non-zero entry at (c, k) means output channel k includes input channel c , weighted by that value.

3.1.1 Identity

The identity modulation returns the $C \times C$ identity matrix for every ensemble member, leaving the input unchanged:

$$\mathbf{M}^{(i)} = \mathbf{I}_C, \quad i = 1, \dots, N_E \quad (3.2)$$

so that $K = C$ and $\mathbf{Y} = \mathbf{X}$, giving the simplest modulation graph, a single diagonal line, shown in Figure 3.1.

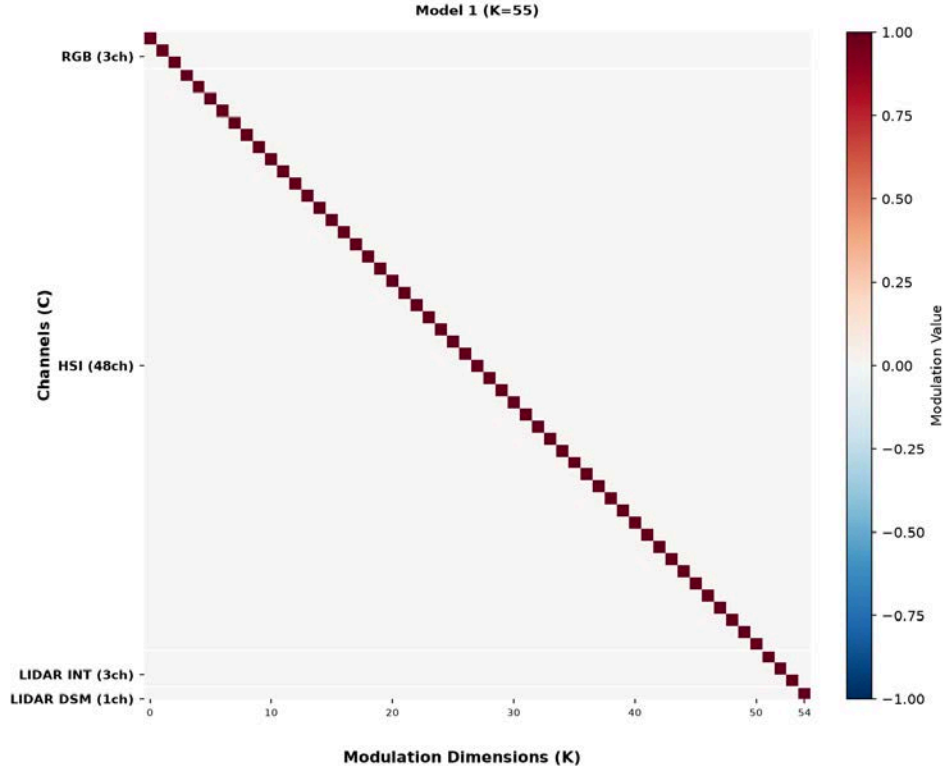


Figure 3.1: *Modulations graph: One of Identity Modulations on DFC18 Dataset*

3.1.2 Random Channel Drop

Let $p \in [0, 1]$ be the **drop probability**. For the i -th ensemble member the modulation matrix $\mathbf{M}^{(i)}$ with $i = 1, \dots, N_E$ is constructed independently as follows. Each channel is kept independently with probability $1 - p$, so in expectation $C \cdot p$ channels are dropped. Define D as

$$D^{(i)}(c) \sim \text{Bernoulli}(1 - p), \quad c = 1, \dots, C. \quad (3.3)$$

Let $S^{(i)} = \{c : D^{(i)}(c) = 1\}$ be the kept channel indices and $K^{(i)} = |S^{(i)}|$.

Reading input channels in their fixed order, the c -th kept channel is mapped to output column $k = c - l$, where l is the number of dropped channels before it: $l = \sum_{c'=1}^{c-1} (1 - D^{(i)}(c'))$. With that define $\mathbf{M}^{(i)}$ as:

$$\mathbf{M}^{(i)}(c, k) = \begin{cases} 1 & \text{if } c \in S^{(i)} \text{ and } k = c - l \\ 0 & \text{else} \end{cases}. \quad (3.4)$$

The modulation graph (Figure 3.2) looks similar to the identity's modulation graph (Figure 3.1), just with some input channels being dropped, since $\mathbf{M}^{(i)}$ is a selection matrix. Because each channel is dropped independently with probability p , the output channel count $K^{(i)}$ can vary per ensemble member. Default parameter: $p = 0.3$.

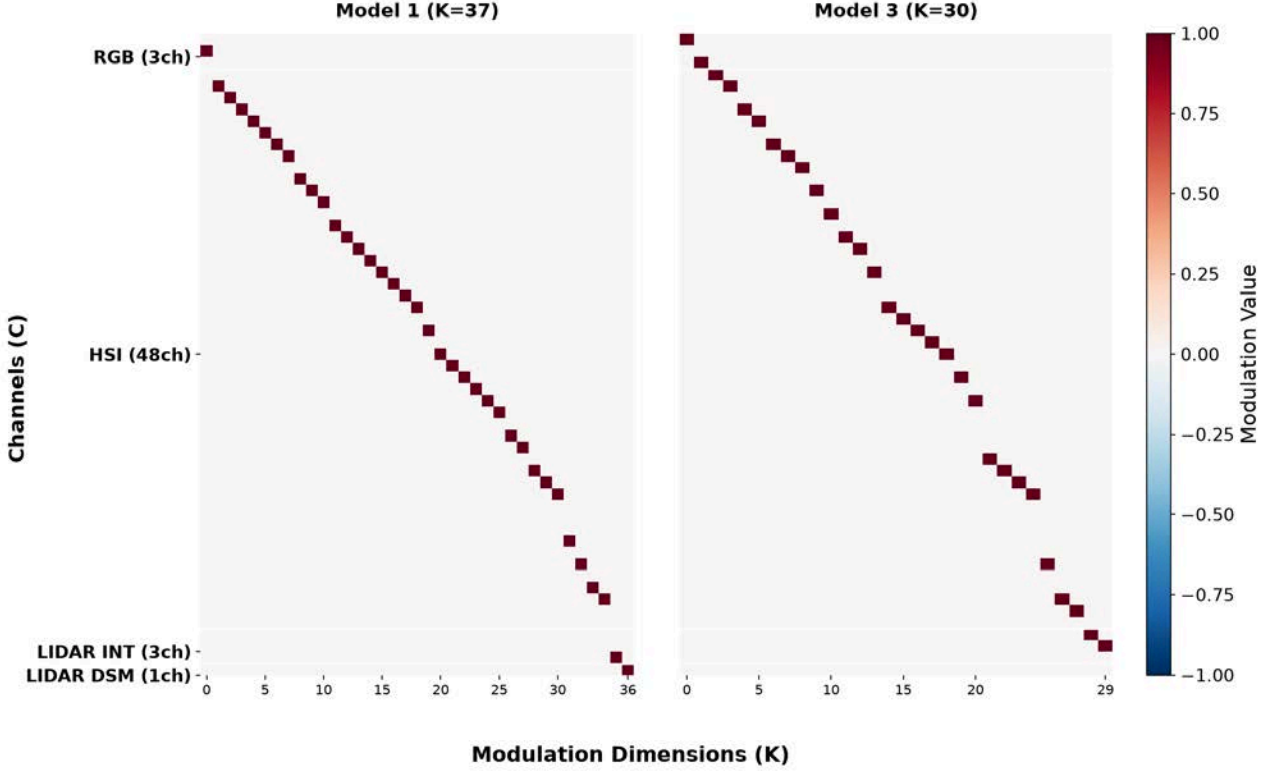


Figure 3.2: Modulations graph: Two of *Random Channel Drop* Modulations on DFC18 Dataset

3.1.3 Random Drop Percentage of Modality

Unlike Random Channel Drop above, which tends to always drop channels from high-channel-count modalities, this modulation drops uniformly across modalities.

One modality m is drawn per ensemble member, $m^{(i)} \sim \text{Uniform}(\mathcal{M})$, where $s_{m^{(i)}}$ denotes the channel index of that modality's first channel and $C_{m^{(i)}}$ its number of channels, so that $[s_{m^{(i)}}, s_{m^{(i)}} + C_{m^{(i)}})$ is its channel index range within $\mathbf{X}$. Only a fixed fraction p of that modality's channels is then dropped:

$$D^{(i)} \subseteq [s_{m^{(i)}}, s_{m^{(i)}} + C_{m^{(i)}}), \quad |D^{(i)}| = \lceil p \cdot C_{m^{(i)}} \rceil. \quad (3.5)$$

All channels outside $D^{(i)}$, including every channel of the other modalities, are kept, so the resulting $\mathbf{M}^{(i)}$ is the selection matrix (as in Random Channel Drop) onto the kept indices $S^{(i)} = \{1, \dots, C\} \setminus D^{(i)}$. Its modulation graph (Figure 3.3) resembles that of Random

Channel Drop when a high-channel modality such as HSI is selected (1st plot). When instead a low-channel modality such as RGB (3 channels, 2nd plot) is selected only a single channel is dropped. Default parameter: $p = 0.3$ with a minimum of 1 dropped channel.

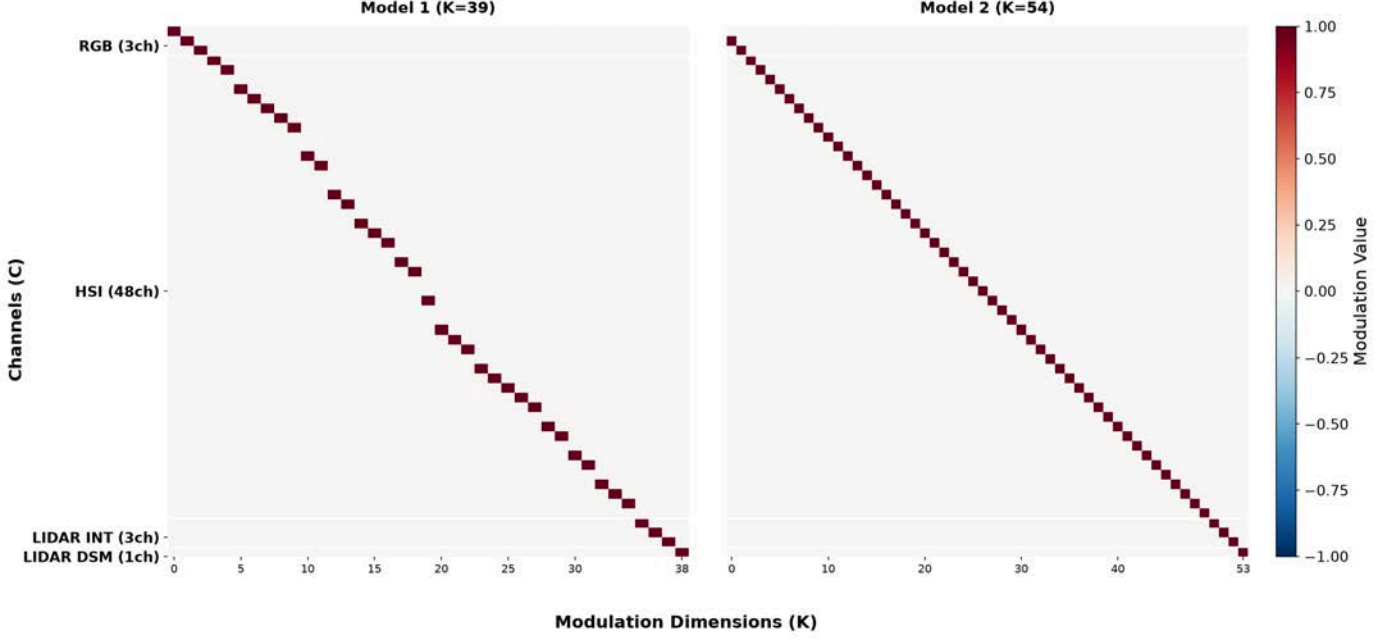


Figure 3.3: *Modulations graph: Two of Random Drop Percentage of Modality Modulations on DFC18 Dataset*

3.1.4 Powerset of all modalities

Given a dataset, this modulation returns all possible combinations of the dataset’s modalities, ignoring the passed modulation count parameter N_E , instead, N_E is computed as follows.

Let $\mathcal{M} = \{m_1, \dots, m_J\}$ be the dataset’s J modalities with channel counts $C_{m_1}, \dots, C_{m_J}$ summing to C and let $\Pi(\mathcal{M})$ denote its powerset excluding the empty set, so $N_E = |\Pi(\mathcal{M})| = 2^J - 1$.

Enumerate its elements as $S^{(1)}, \dots, S^{(N_E)} \in \Pi(\mathcal{M})$.

For each $S^{(i)}$ define $K^{(i)} = \sum_{m \in S^{(i)}} C_m$.

The associated modulation $\mathbf{M}^{(i)} \in \{0, 1\}^{C \times K^{(i)}}$ is a **selection matrix**: reading input channels in their fixed dataset order, the c -th channel belonging to modality m_i in $S^{(i)}$ is mapped to output column $k = c + l$, where l is the sum of the “previous” channel counts: $l = \sum_{i=1}^{c-1} C_{m_i}$. Therefore define $\mathbf{M}^{(i)}$ as

$$\mathbf{M}^{(i)}(c, k) = \begin{cases} 1 & \text{if } c \text{ is a channel of modality } m \text{ in } S^{(i)} \text{ and } k = c + l \\ 0 & \text{else} \end{cases} \quad (3.6)$$

The example modulations graph (Figure 3.4) shows 9 out of the total $\Pi(4) = 15$ modulations for brevity. Each graph shows a different combination of modalities, with exactly one nonzero entry per row and column, dropping all channels of modalities not in $S^{(i)}$. These entries lie mostly on the diagonal when high-channel modalities such as HSI are included.

3.1.5 Principal Component Analysis (PCA)

Unlike the previous modulations, which are either fixed selection matrices or random projections independent of the data, PCA modulation runs a standard Principal Component Analysis. It finds the principal components of the given data to form, per ensemble member, linear projections along these directions.

Each output channel is a direction of maximum variance in the channel space, estimated from a random subset of the training data.

Let $p \in (0, 1]$ be the **subset fraction**. For each ensemble member $i = 1, \dots, N_E$, a subset of tile indices is drawn without replacement, $\mathcal{J}^{(i)} \subseteq \{1, \dots, n\}$, $|\mathcal{J}^{(i)}| = n_s = \lceil p \cdot n \rceil$, where n is the number of tiles in the dataset.

Treating every one of the $H \cdot W$ pixels per n_s sampled tiles across all channels C as observations in $\mathbb{R}^C$ gives the **data matrix** $\mathbf{X}^{(i)} \in \mathbb{R}^{(n_s HW) \times C}$.

Each channel is centered by subtracting its mean over all sampled pixels. Writing $\mathbf{x}_r \in \mathbb{R}^C$ for the r -th pixel of $\mathbf{X}^{(i)}$ and let $\tilde{\mathbf{X}}^{(i)}$ be the matrix of centered rows $\tilde{\mathbf{x}}_r$ to define $\boldsymbol{\mu}$ and $\tilde{\mathbf{x}}$ as follows:

$$\boldsymbol{\mu} = \frac{1}{n_s HW} \sum_{r=1}^{n_s HW} \mathbf{x}_r \quad \text{and} \quad (3.7)$$

$$\tilde{\mathbf{x}}_r = \mathbf{x}_r - \boldsymbol{\mu} \quad . \quad (3.8)$$

The (reduced) singular value decomposition of $\tilde{\mathbf{X}}^{(i)}$ is computed as:

$$\tilde{\mathbf{X}} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T \quad \text{where} \quad \boldsymbol{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_C), \quad \sigma_1 \geq \dots \geq \sigma_C \geq 0 \quad . \quad (3.9)$$

The columns $\mathbf{v}_1, \dots, \mathbf{v}_C \in \mathbb{R}^C$ of $\mathbf{V}$ are the orthonormal eigenvectors of the channel covariance matrix $\tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$, ordered by decreasing explained variance. The modulation matrix for ensemble member i is formed from the first K of them, so the projection onto the K -dimensional principal subspace:

$$\mathbf{M}^{(i)} = (\mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_K) \in \mathbb{R}^{C \times K} \quad . \quad (3.10)$$

K is set in practice either directly (components_count) or as a percentage of C (components_percentage), or as that percentage capped at a fixed maximum (max_components_count).

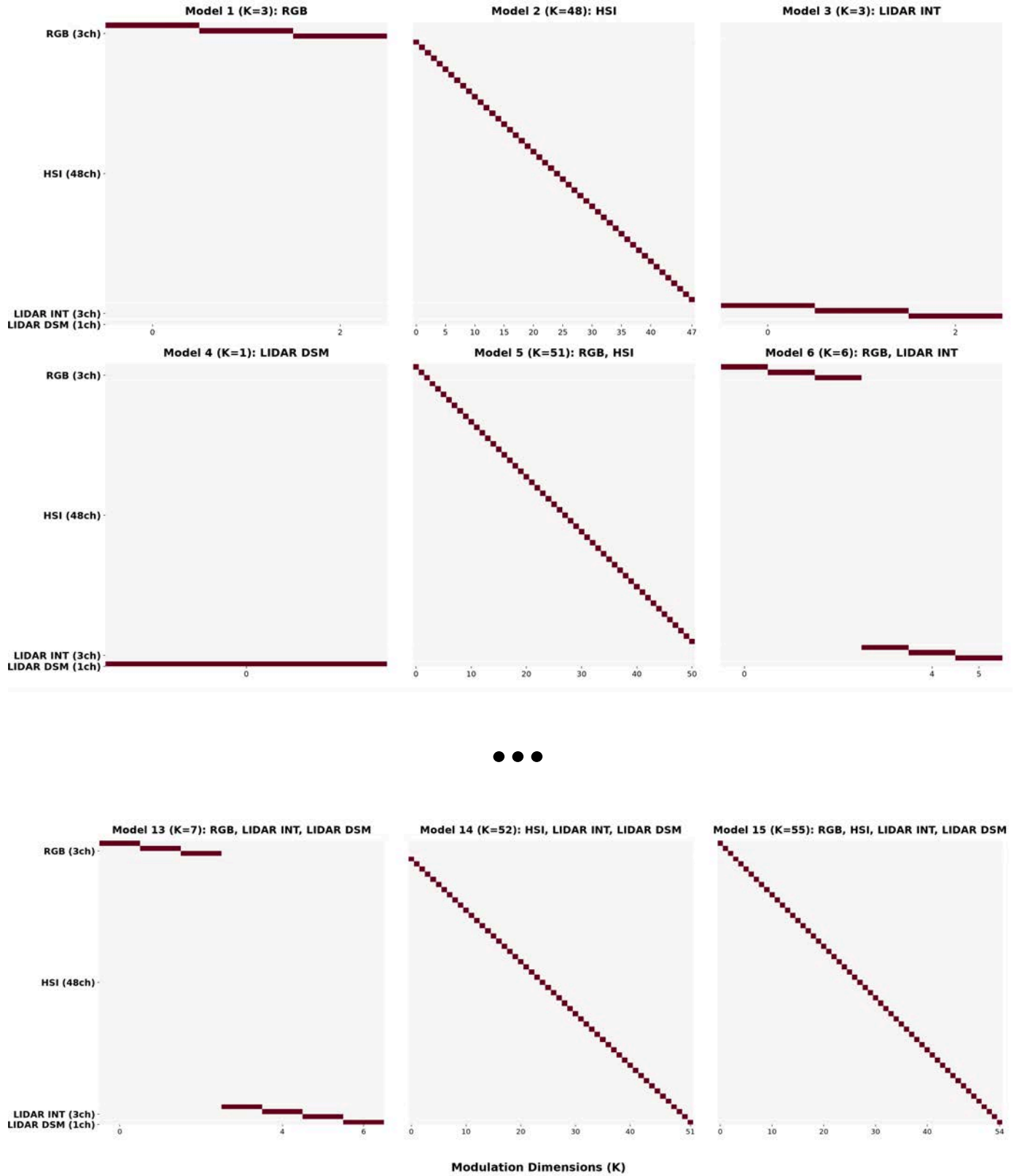


Figure 3.6: *Modulations graph: Subset of all 15 Powerset of all modalities Modulations on DFC18 Dataset*

The modulations graph (Figure 3.7) shows 2 modulations, both exhibiting a very similar pattern going from very high (or very low) values in low-channel modalities RGB, LiDAR DSM, LiDAR Int to high-channel modality HSI, in that order. Output channel 0 is constantly absolute high valued, since in the dataset a high value (bright pixel) in one channel is heavily correlated with the other channels also being higher. Here sign of the values don't matter, they represent the same eigenvector, so the same counts for low values. The small differences besides the sign come from the differing selected subset of samples. Default parameters: $K = \text{all channels}$ and $p = 0.75$.

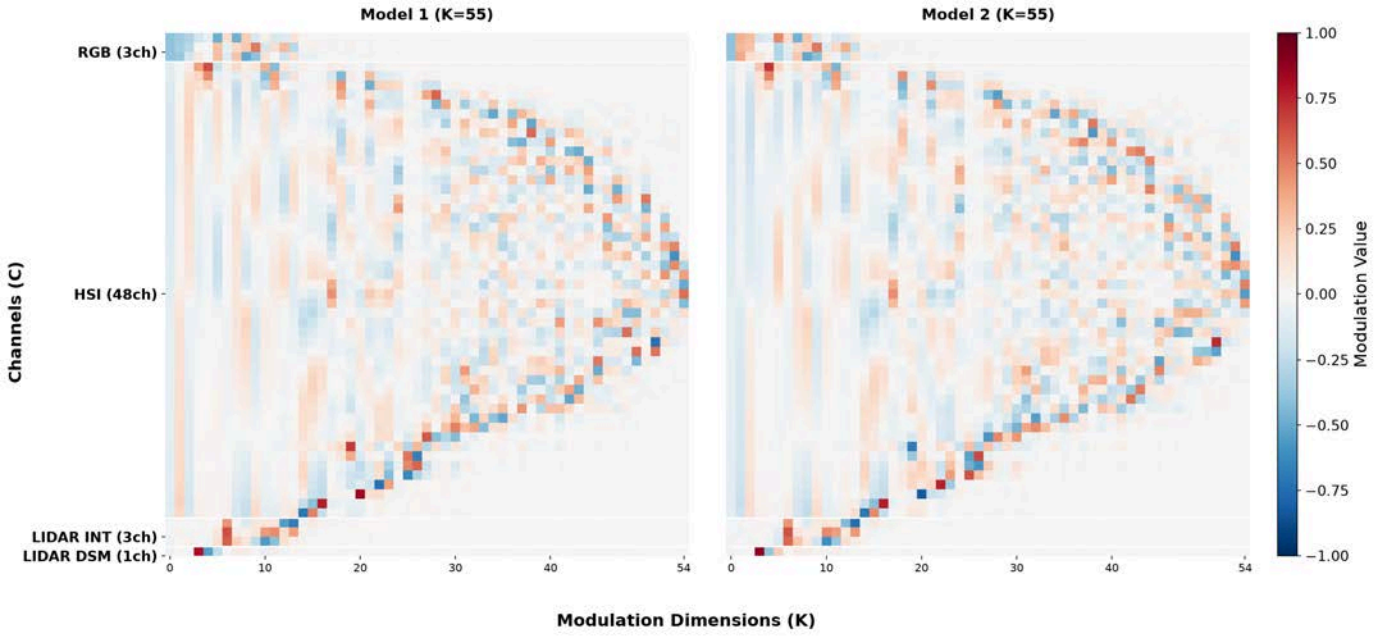


Figure 3.7: Modulations graph: Two of PCA Modulations on DFC18 Dataset

As a diagnostic, the **explained variance ratio (EVR)** is used:

$$\text{EVR}(K) = \frac{\sum_{k=1}^K \sigma_k^2}{\sum_{k=1}^C \sigma_k^2} . \quad (3.11)$$

It gives the fraction of total channel variance of member i 's subset retained by the chosen K , where a value close to 1 indicates that little information is discarded by the projection.

Modulation chaining

An optional step adds further diversity across ensemble members before PCA is fit: PCA can be chained after another channel modulation $\mathbf{M}_{\text{prev}}^{(i)} \in \mathbb{R}^{C \times K_{\text{prev}}}$ (previous_modulation). The data matrix is then built from the prev-modulated tiles, with C replaced by K_{prev} and the final modulation is the composition of both linear maps:

$$\mathbf{M}_{\text{final}}^{(i)} = \mathbf{M}_{\text{prev}}^{(i)} \mathbf{M}^{(i)} \in \mathbb{R}^{C \times K} . \quad (3.12)$$

Because each ensemble member sees an independently drawn subset of tiles (and optionally a different previous modulation), the fitted principal directions differ across members, giving the ensemble its diversity while each individual modulation remains the locally variance-maximizing projection, in contrast to the other random projections above.

3.1.6 Random Linear Combination

Let $\mathbf{W} \in \mathbb{R}^{C \times K}$ have independent and identically distributed random entries $W_{c,k} \sim \text{Uniform}(0, 1)$.

For each ensemble member $i = 1, \dots, N_E$, the modulation matrix $\mathbf{M}^{(i)}$ is constructed independently, with each column normalized to sum to one:

$$\mathbf{M}^{(i)}(c, k) = \frac{\mathbf{W}(c, k)}{\sum_{c'=1}^C \mathbf{W}(c', k)} . \quad (3.13)$$

As seen in the modulations graph (Figure 3.8), every output channel k is a random combination of all C input channels, where $\sum_{c=1}^C \mathbf{M}^{(i)}(c, k) = 1$.

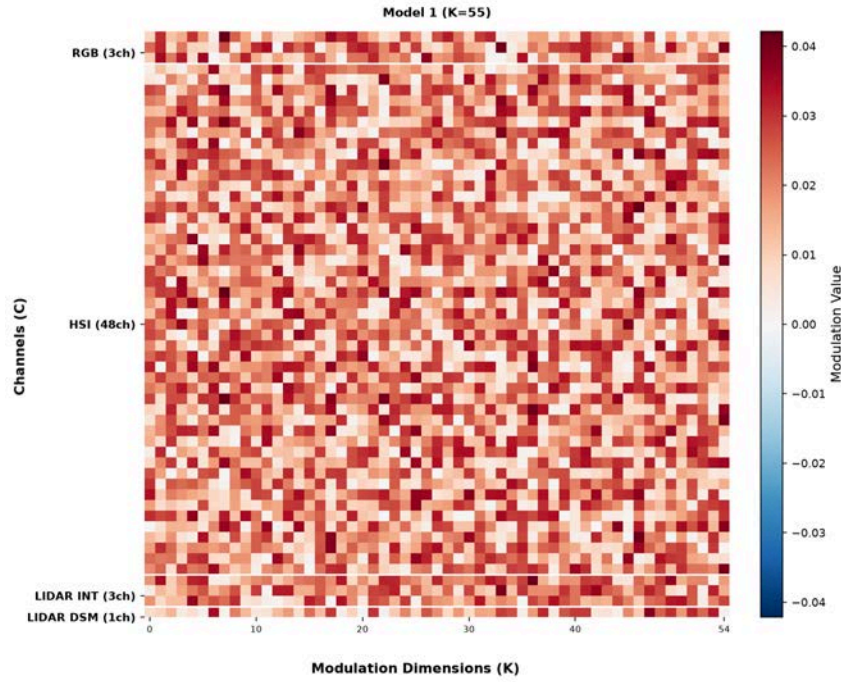


Figure 3.8: *Modulations graph: One of **Random Linear Combination** Modulations on DFC18 Dataset with a stretched colormap for better readability*

3.1.7 Random Classes Simple LDA

A simplified, stochastic variant of Linear discriminant analysis (LDA): instead of the true between/within-class scatter matrices, each projection direction is the difference of

two randomly selected class samples. For each output channel $k = 1, \dots, K$ of ensemble member i :

1. Draw two class labels $A, B \sim \text{Uniform}(\mathcal{C})$ from the dataset's class set $\mathcal{C}$, excluding the case $A = B$.
2. Draw one pixel of class A and one of class B uniformly at random from the reference data masks. This gives channel vectors $\mathbf{x}_A, \mathbf{x}_B \in \mathbb{R}^C$ at those pixel locations.
3. Set column k to their difference:

$$\mathbf{M}^{(i)}(\cdot, k) = \mathbf{x}_A - \mathbf{x}_B \quad . \quad (3.14)$$

Repeating this K times per ensemble member yields $\mathbf{M}^{(i)} \in \mathbb{R}^{C \times K}$, a random projection biased towards directions that separate sampled class pairs.

The modulations graph (Figure 3.9) shows, in each column (output channel), the difference between two randomly selected pixels of different classes as vertical stripes. Default parameter: $K = \text{all channels}$.

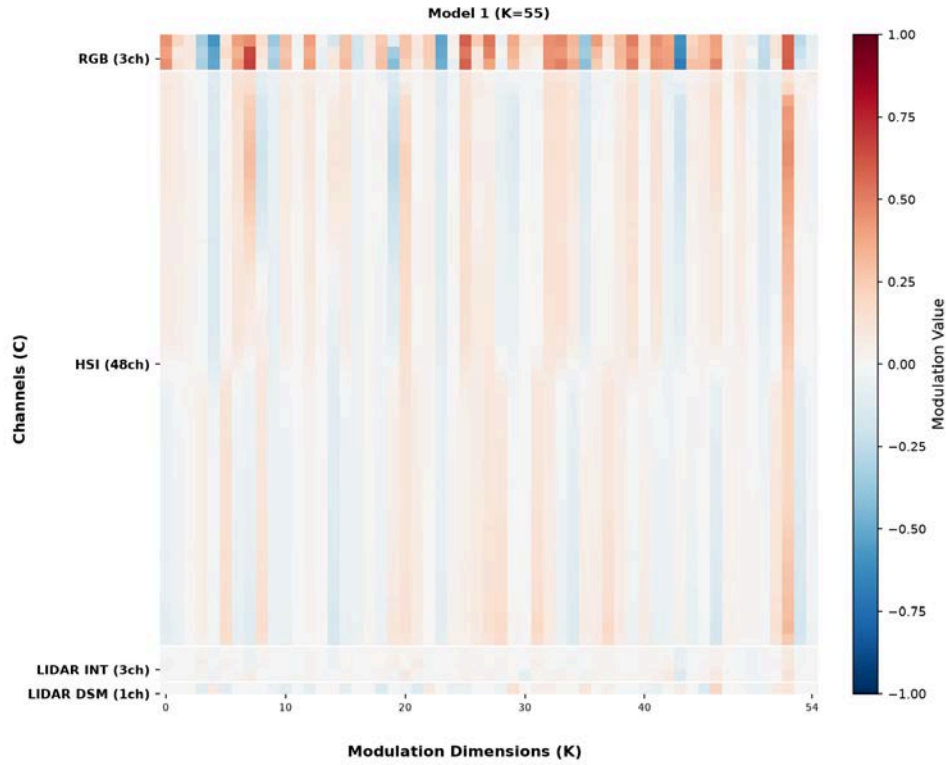


Figure 3.9: *Modulations graph: One of Random Classes Simple LDA Modulations on DFC18 Dataset*

3.2 Ensembles

Let $f : m \rightarrow \mathbb{R}^{N_E \times C \times K}$ denote a **Channel Modulation Function**. It returns N_E modulations $\mathbf{M} = \mathbf{M}^{(1)}, \dots, \mathbf{M}^{(N_E)}$, as described in more detail above in Section 3.1 and therefore sets the size N_E of an ensemble. An ensemble is defined by its channel modulation function f , which in turn defines its ensemble members, each of which is constructed as in Section 3.3 below.

3.2.1 Prediction

Once all models of an ensemble are trained, the ensemble as a whole can produce a prediction. Although numerous ways exist to aggregate separate model predictions into one, for the sake of simplicity the ensemble here takes the average of the models predictions while including their certainty.

This works as follows: To get prediction probabilities from the neural network models output unnormalized prediction matrix of **prediction class logits** $\mathbf{P} \in \mathbb{R}^{N_E \times N_C \times N}$, softmax (Section 3.2.1.1) is used, which turns them into **class probabilities** $\mathbf{Q} \in \mathbb{R}^{N_E \times N_C \times N}$. It is necessary to use the class probabilities formed this way rather than the original class logits since those are not constrained, so one model could have higher impact on the decision than another one just by having larger absolute logits values. Then for the majority voting taking the mean of $\mathbf{Q}$ gives the **mean class probabilities** $\mathbf{R} \in \mathbb{R}^{N_C \times N}$. To achieve the final prediction the argmax function is used, which simply gives the class with the highest probability for each pixel as the **final ensemble prediction** $\mathbf{K} \in \mathbb{R}^N$.

Softmax The Softmax function does not change the relative ordering of class probabilities, it just normalizes them to $[0, 1]$ and gives higher logits an exponentially higher probability than lower logits of the same model. For each models output class logits $x_i = \mathbf{P}(\cdot, i)$ the class probabilities $\hat{x}_i = \text{Softmax}(x_i)$ are calculated as follows:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} . \quad (3.15)$$

3.2.2 Testing

All performance metrics introduced in Section 4.2 are computed to rate the performance of ensembles with their specific modulation and to enable comparison with other works.

When showing the ensembles prediction $\mathbf{K}$ as label maps on the whole testset like in Figure 4.13, the Shannon Entropy (Section 3.2.2.1) is also computed to show how confident the ensembles are with their prediction at each pixel.

Shannon Entropy The Entropy H is low in a pixel where one class dominates (confident), high where probability is spread evenly across classes (uncertain). By using $\log_{N_C}$ it is scaled to $[0, 1]$ and reaches 0 when one class has probability 1. It is calculated on the mean class probabilities $\mathbf{R} \in \mathbb{R}^{N_C \times N}$ with $p_c = \mathbf{R}(c, i)$ for a class c and given pixel index i :

$$H = - \sum_c p_c \cdot \log_{N_C}(p_c) \quad . \quad (3.16)$$

3.3 Ensemble Members

An ensemble member is defined by its channel modulation $\mathbf{M}^{(i)}$, a single modulation drawn from the modulation function's output $\mathbf{M}$. This $\mathbf{M}^{(i)}$ is what makes each member a distinct predictor: Before the neural network sees the data, it is transformed by applying $\mathbf{M}^{(i)}$, so as formulated in Section 3.1, the C channels of the dataset become the K modulated channels fed to the model.

The standard **UNet architecture** from [4] is shown in Figure 3.10. It is a type of Convolutional Neural Network (CNN) with a downsampling and an upsampling path, connected by skip connections that concatenate feature maps from the former to the latter. Every layer performs a **Double Convolution**, consisting, in order, of Convolution, Batch Norm, ReLU, Convolution, Batch Norm and ReLU.

Batch Norm is optional here, but helpful for stability whenever the data is not consistently normalized between 0 and 1, as is the case when a modulation such as PCA (Section 3.1.5) is used. Without it, the gradient of the loss function can become so steep that the loss diverges and becomes NaN.

In the downsampling path, the Double Convolution is followed by a **Max-Pool** operation. In the upsampling path, it is preceded by an **Up-Convolution**, which is a convolution operation coined in [4] that doubles the spatial resolution while halving the channel count.

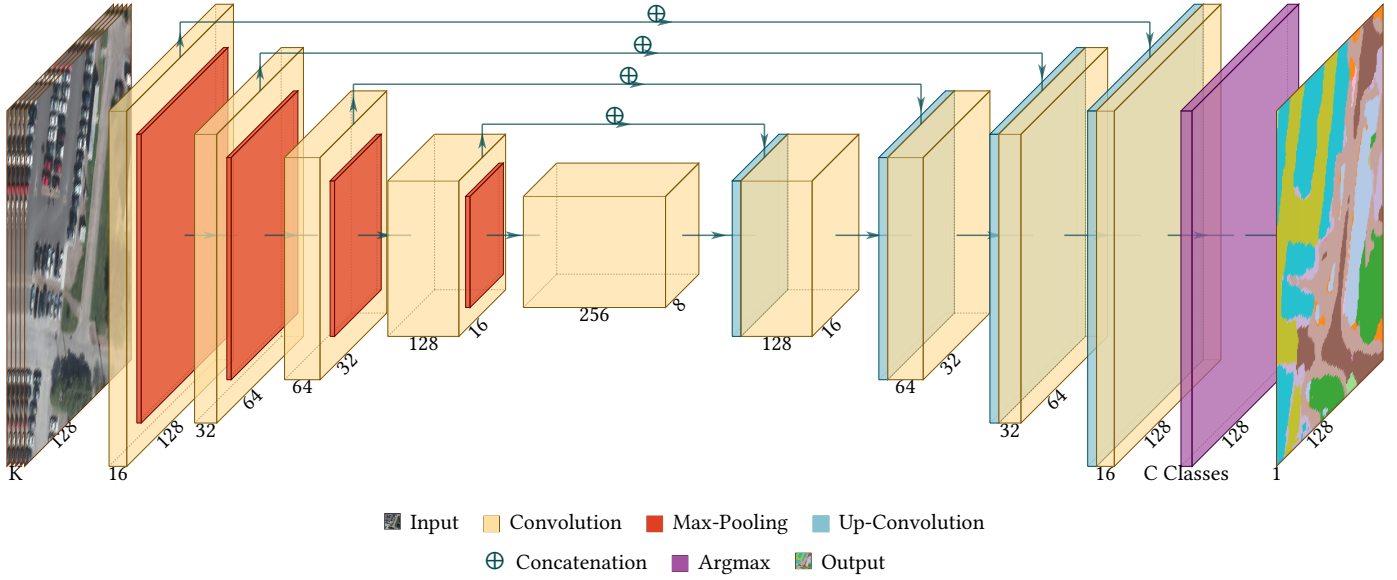


Figure 3.10: U-Net architecture used in this work with the layer sizes. The skip connections are visible as the concatenations above the graph

During training **random transformations** are applied to the data to produce more varied data tiles, which generally improves performance. Here, these transformations are limited to random horizontal and/or vertical flipping. Many further transformations could be applied, but discussing them in more depth would be beyond the scope of this work.

In general, the **model is not optimized to its full potential**, numerous ways exist to improve it as a base learner, but doing so is not the goal of this thesis, since the base learner is only ever compared against itself, with and without modulated ensemble learning. To at least “approach” results reported in related work, the model still uses a few common, simple tweaks, such as random transformations, an adaptive learning rate and early stopping.

3.3.1 Training

The optimizer used during training is the standard **Adam Optimizer with weight decay (AdamW)**, applied to the **Cross Entropy Loss** function (Section 3.3.1.1) over all classes of the dataset.

Early stopping is implemented, so training stops once the validation loss has not decreased for a certain number of epochs, at which point the best model parameters observed so far are used. An **adaptive learning rate** is likewise applied via an LR-Scheduler, decreasing the learning rate by a percentage whenever a certain number of epochs pass without validation loss improvement.

In addition to the loss on the training set (training loss) and on the validation set

(validation loss), the DICE score is computed on both sets, yielding a training DICE and a validation DICE.

Cross Entropy Loss This loss, computed during training, forms the basis for model optimization, since the model’s backward pass is derived from it. It accounts for the class weights w and averages the losses over all classes $\neq 0$ and is defined as:

$$l(x, y) = \sum_{n=1}^N \frac{1}{\sum_{n=1}^N w_{y_n} \cdot \mathbf{1}\{y_n \neq 0\}} l_n . \quad (3.17)$$

3.3.2 Class weights

The class weights $W \in \mathbb{R}^{N_C}$ are computed from the **class counts** $\Gamma \in \mathbb{R}^{N_C}$, with Γ_c denoting how often class c appears in the dataset and $N_C = |\mathcal{C}|$. Since counting all pixels of the entire reference data mask takes a few seconds, this is done only once per dataset and the result is cached to disk. The weight for class 0 is always 0.

Class weights are used to account for **class imbalance** in the dataset. In the *DFC18* dataset for example, class 2 “*Stressed Grass*” appears 100,660 times in the reference data mask, while class 17 “*Unpaved Parking Lots*” appears only 587 times, so 171 times less often.

A simple way to counter this imbalance is to use **Inverse Frequency Weights**, with a small $\varepsilon = 10^{-6}$ to avoid division by zero and letting $\text{norm}(X) = \frac{X}{\sum X}$. The weights vector W is given by:

$$w_c = \frac{1}{\Gamma_c + \varepsilon}, \quad w_0 = 0, \quad W = \text{norm}([w_0, \dots, w_C]) . \quad (3.18)$$

This, however, takes the imbalance into full account and effectively equalizes all classes, which is not entirely correct either: the training data simply reflects that some classes occur more often than others by nature and if the dataset is well constructed, the test set follows a similar pattern.

Each w_c is therefore **clamped** between a minimum value of 0.25 and a maximum value of 4 before normalization, values that were determined empirically:

$$w_c = \begin{cases} 0.25 & | \ w_c < 0.25 \\ 4 & | \ w_c > 4 \\ 0 & | \ c = 0 \\ w_c & \text{else} \end{cases} . \quad (3.19)$$

```
→ src/UNet.py
   src/ModelManager.py
→ src/train.py
→ src/class_counts_weights.py
```

Experiments

The methodology is applied to two datasets, comparing ensembles using specified performance metrics as laid out below. First, a baseline is established without modulation for each dataset, simply training an Ensemble of UNets on the full dataset. This baseline is then compared against results that use modulation, with varying hyperparameters for multiple modulations.

4.1 Datasets

4.1.1 Dataset of the 2018 IEEE GRSS Data Fusion Contest (DFC2018)

This dataset is introduced in “Advanced Multi-Sensor Optical Remote Sensing for Urban Land Use and Land Cover Classification” [5]. Also known as DFC2018, it was acquired by NCALM at the University of Houston (UH) on February 16, 2017, covering the UH campus and its surrounding urban area and comprises four modalities depicting the same area:

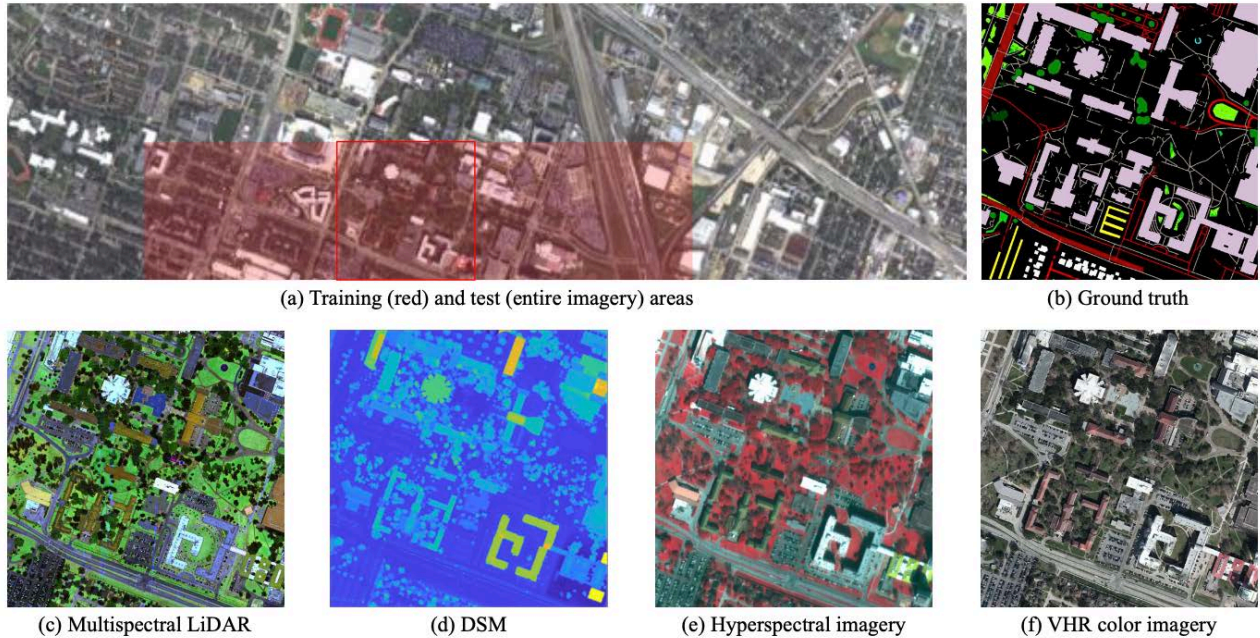


Figure 4.1: Image and description taken from [5] showing the DFC18 dataset overview. (a) Training (red) and test (entire imagery except red) areas, examples of (b) reference data, (c) color composite of multispectral LiDAR intensity, (d) DSM, (e) color composite of hyperspectral imagery and (f) VHR color imagery.

- **RGB**: very-high-resolution (VHR) color imagery at 0.05 m GSD spectral range, at 1 m GSD
- **LIDAR**: multispectral LiDAR intensity, three wavelengths (1550 nm, 1064 nm near-infrared, 532 nm green), at 0.5 m GSD
- **DSM**: LiDAR-derived digital surface model (elevation), at 0.5 m GSD
- **HSI**: hyperspectral imagery, 48 wavelengths between 380-1050 nm

This gives a total channel count of $C = 55$ (RGB: 3, HSI: 48, LIDAR: 3, DSM: 1).

The reference data covers 20 land use / land cover classes, plus an “Unclassified” class 0 for unlabeled pixels that is ignored both during training and for all metrics. Since classes are heavily imbalanced in the training area, class weights are used, as described in Section 3.3.2.

Train and test are two disjoint regions of the dataset rather than a random pixel split, so evaluation happens on a fully unseen area. The organizers additionally resampled the test reference data toward a more balanced class distribution.

The data and reference data must first be prepared for use with the deep neural network model:

1. **Scale** to the reference data’s 0.5m GSD: down-sample RGB ($0.05 \rightarrow 0.5$ m) and up-sample HSI ($1 \rightarrow 0.5$ m) using *cubic* scaling, normalizing both while scaling
2. **Merge**: Stitch the separately delivered RGB tiles into one contiguous raster covering the full scene
3. **Layer**: Stack the three LiDAR intensity channels into one 3-channel raster, normalizing per channel
4. **Correct**: Replace sensor NoData values of LIDAR & DSM with NaN and remove the non-spectral bands 48 and 49 from the HSI imagery
5. **Cut**: Crop HSI, LIDAR and DSM to the extent of the training reference data
6. **Tile**: Open all modalities and the reference data together and split them into aligned $\text{SIZE} \times \text{SIZE}$ patches ($\text{SIZE} = 128$), so that patch i covers the same ground area across every modality and the mask, while skipping edge tiles that have less than 30% data content

The tile size $\text{SIZE} = 128$ is the standard size for UNets, since it is the input layer size of the standard net. This approach yields 368 training and 1,234 testing tiles.

```
→ src/prep_merge_cut_layer_norm.py,
   src/prep_scale_editGT.py
   src/prep_tile.py
→ src/DFC18_Dataset.py,
   prep_DFC18.py
```

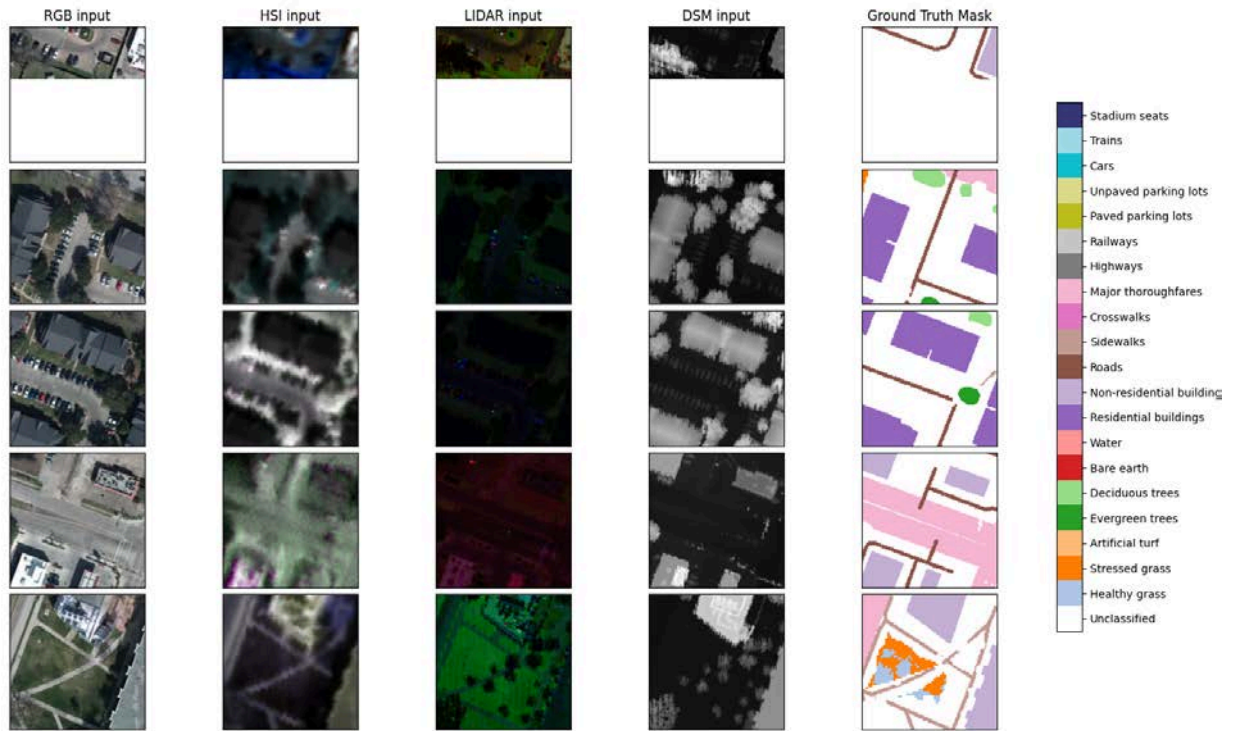


Figure 4.2: DFC18: 5 Example tiles from the trainset

4.1.2 MDAS Dataset

This dataset, introduced in the paper “MDAS: A New Multimodal Benchmark Dataset for Remote Sensing” [1], referred to as MDAS, covers the city of Augsburg, Germany ($\approx 121.7 \text{ km}^2$). Except for the reference data, all remote sensing data were acquired on the same day, May 7, 2018. But unlike DFC2018, whose four modalities were all flown together for the contest, MDAS combines five sensors of very different origin and native resolution, ranging from an airborne camera system to satellite instruments:

→ `src/MDAS_Dataset.py`,
`prep_MDAS.py`



Figure 4.3: Image and description taken from [1] showing the MDAS dataset overview. From left to right: the Sentinel-1 image (VH band in dB), the Sentinel-2 image (RGB: 665, 560, 490 nm), the OSM land use layer, the HySpex image (RGB: 691, 569, 445 nm) and the DSM.

- **SAR:** Sentinel-1 synthetic aperture radar, VV/VH intensities (2 channels), at 10m GSD
- **RGB:** VHR color imagery from the DLR 3K airborne camera system, at 0.2m GSD
- **MSI:** Sentinel-2 multispectral imagery, 12 bands (440-2200 nm), all bands upsampled to 10m GSD
- **HSI:** Hyperspectral imagery, 242 bands, at 10m GSD
- **DSM:** Digital surface model (elevation), resampled to 0.3m GSD

This gives a total channel count of $C = 260$ (RGB: 3, HSI: 242, MSI: 12, SAR: 2, DSM: 1).

MDAS does include a semi-manually labeled reference, but it is of poor quality for land cover classification, since it covers only three small subregions of the whole dataset and was made for a different task: hyperspectral unmixing. A custom dataset is therefore created from the MDAS data, with the following considerations.

Since no expert-labeled reference exists for the whole scene (unlike DFC2018), the reference data used here instead comes from OpenStreetMap (OSM) building, land use and water maps, rasterized and remapped into 7 classes (*Forest, Residential, Commercial, Low Plant, Allotments, Recreation Ground, Water*), plus an “Unclassified” class 0 that is again ignored both during training and for all metrics. Classes are heavily imbalanced here too: “*Residential*”, for example, makes up about 6.51 million labeled pixels against only 184,442 for “*Recreation Ground*”, roughly 35 times fewer.

MDAS ships as a single scene without an official train/test split, so tiles are instead split randomly after tiling: 50% for training, 50% for testing, using a fixed random seed so that class counts and class weights w stay reproducible. The more common dataset split of 80%/20% could not be used in these experiments because training some models could take longer than the maximum GPU reservation time allows.

The data and reference data must likewise be prepared for use with the deep neural network model:

1. **Combine:** Merge the OSM water map on top of the OSM land use map into a single GT map
2. **Reclassify:** Remap the raw OSM tag codes to the 7 target classes (skipping and merging some, e.g. several land use subtypes into *Low Plant* or *Commercial*), plus 0 for unlabeled pixels
3. **Cut:** Crop the combined reference data by 1px at the edge, since OSM maps have an unclear GSD of 2.2001982161m, to obtain a cleaner GSD of 2.2m
4. **Scale:** Rescale RGB, MSI, SAR and DSM to 2.2m GSD, normalizing while scaling. HSI is left unscaled and only normalized in place, since reprojecting the full cube would be too large (≈ 24 GB)
5. **Correct:** Remove the fourth band from the RGB imagery so that it matches RGB's standard 3-channel format
6. **Tile:** Open all modalities and the reference data together and split them into aligned $\text{SIZE} \times \text{SIZE}$ patches ($\text{SIZE} = 128$), streaming HSI from disk instead of loading the full cube into memory and skipping edge tiles that have too little data content
7. **Split:** Randomly assign 50% of the resulting tiles to train/test

This approach yields both 784 training and testing tiles.

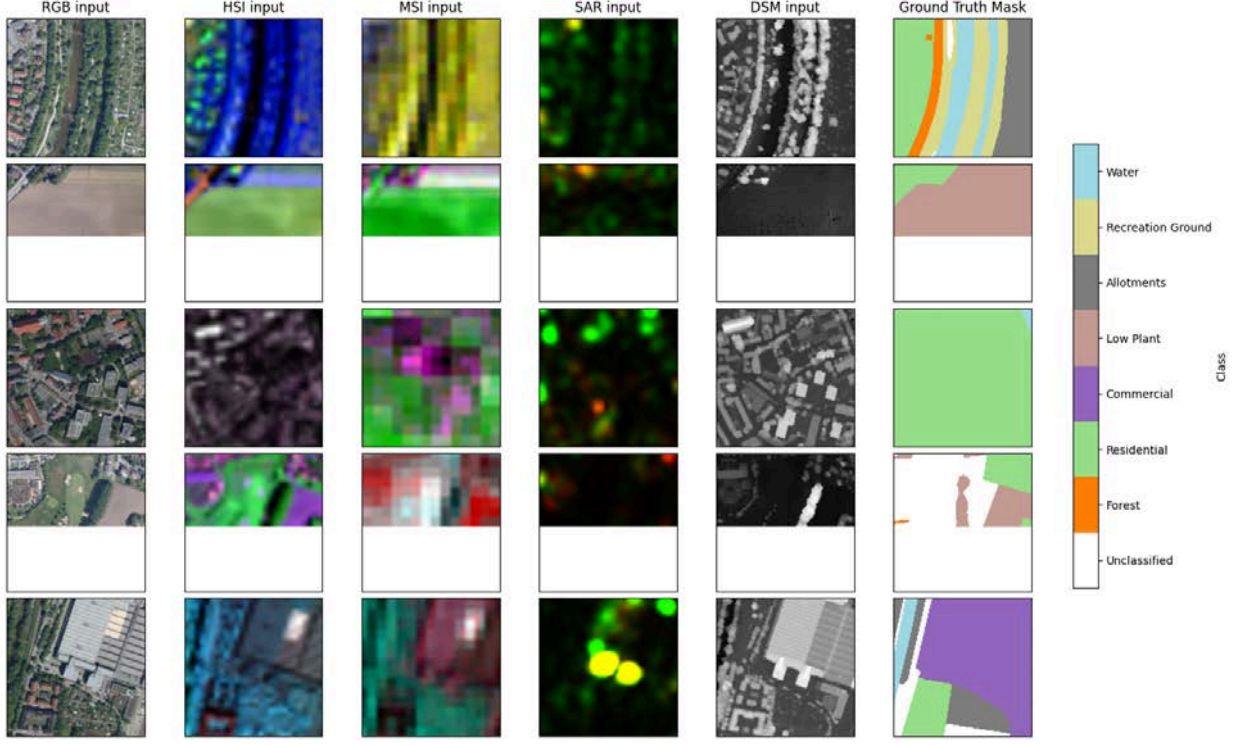


Figure 4.4: MDAS: 5 Example tiles

4.2 Performance Metrics

The metrics used in the experiments are laid out below.

All of them compare a **prediction mask** $\mathbf{P}$ with a **reference data mask** $\mathbf{Y}$, while ignoring the unclassified class (class $0 \in \mathcal{C}$) in the reference data masks, since inference is also of interest on these **samples** (= **pixels**) so the model should not be penalized for predicting on unlabeled samples. Further define the class count $N_C = |\mathcal{C}|$, the spatial dimensions of an image $W = H$ and $N = W \cdot H$.

Taking the **argmax** of the prediction results $\mathbf{R} \in \mathbb{R}^{N_C \times W \times H}$ output by the model, along its first dimension, gives the prediction mask $\mathbf{P} \in \mathbb{R}^{W \times H}$, in the same shape as the reference data mask $\mathbf{Y} \in \mathbb{R}^{W \times H}$.

Furthermore, define the mask $\mathbf{P}_v$ as $\mathbf{P}$ reduced to only its **valid samples**, samples where the reference data mask $\mathbf{Y} \neq 0$, with all other samples in $\mathbf{P}_v$ set to 0. Consequently, $\mathbf{Y} = \mathbf{Y}_v$, since all samples of class 0 are already set to 0.

Let $\mathbf{Q} \in \mathbb{R}^{N_C \times N_C}$ be the **confusion matrix** of the dataset: every valid sample from the reference data $\mathbf{Y}_v$ and prediction mask $\mathbf{P}_v$ is taken and accumulated into $\mathbf{Q}$ as follows:

$$Q_{i,j} = \text{count of samples with true class } i \text{ and predicted class } j \quad (4.1)$$

→ src/metrics.py

To compute the metrics that follow, the confusion matrix $\mathbf{Q}$ is used to count

samples per class c in Y as $n_y^c = \sum_i^{N_C} Q_{c,i}$,
correctly predicted samples per class c as $n_{\top}^c = \sum_i^{N_C} Q_{c,c}$,
predicted samples per class c as $n_p^c = \sum_i^{N_C} Q_{i,c}$ and
correct samples as $n_{\top} = \sum_i^{N_C} Q_{i,i}$.

4.2.1 Accuracies

Overall Accuracy (OA) OA is the fraction of correctly classified samples $n_{\top}$:

$$OA = \frac{n_{\top}}{N} . \quad (4.2)$$

Average Accuracy (AA) AA instead first computes the accuracy per class A^c and then averages across classes, using the class count $N_C = |\mathcal{C}|$:

$$A^c = \frac{n_{\top}^c}{n_p^c} \quad \text{and} \quad (4.3)$$

$$AA = \frac{\sum_c A^c}{N_C} . \quad (4.4)$$

Since OA is dominated by frequent classes while AA weighs every class equally, the gap between the two indicates how much performance depends on class frequency.

4.2.2 Cohen's Kappa

Cohen's Kappa is defined as:

$$\kappa = \frac{p_o - p_e}{1 - p_e} . \quad (4.5)$$

where p_o is the observed probability of agreement and p_e is the expected agreement when all class labels are assigned randomly according to their marginal frequencies (how often each class relatively appears in the (test) set). In other words, p_e is the probability that a true-class- c_1 sample and a predicted-class- c_2 sample would coincide by chance alone, summed over all classes.

Kappa provides information beyond OA: if one class covers 90% of samples, a model that predicts only that class everywhere achieves a high observed agreement of $p_o = 0.9$, but p_e is also close to 0.9, so $\kappa \approx 0$, correctly showing that the model has learned nothing beyond random guessing.

Overall Accuracy (OA), in comparison, cannot detect this, since it does not account for the predicted class distribution, only whether each sample was right or wrong.

The steps to calculate p_o and p_e are as follows.

The **observed agreement** p_o is simply the normalized sum of the diagonal of the confusion matrix $\mathbf{Q}$ and is therefore the same as the Overall Accuracy (OA):

$$p_o = \frac{n_{\top}}{N} . \quad (4.6)$$

The **expected agreement** p_e uses how often each class truly occurs, n_y^c and how often each class is predicted, n_p^c :

$$p_e = \sum_c \frac{n_y^c}{N} \cdot \frac{n_p^c}{N} . \quad (4.7)$$

4.2.3 Multiclass DICE Score

DICE is a similarity measure over two sets A and B , representing the ratio of agreed samples to total samples. For a binary mask, it is defined as follows:

$$\text{DICE}(A, B) = \frac{2 \cdot |A \cap B|}{|A| + |B|} . \quad (4.8)$$

Since the model addresses a multi-class classification problem, the multiclass DICE score is defined as the list of scores over all classes $c \in \mathcal{C}$.

For each class c , define $\mathbf{P}^c$ and $\mathbf{Y}^c$ as the valid masks $\mathbf{P}_v$, $\mathbf{Y}_v$ restricted to mask value = c , with all other classes set to 0.

Using these class- c -reduced valid masks, the multiclass DICE is defined as:

$$\overline{\text{DICE}} = [\text{DICE}(\mathbf{P}^c, \mathbf{Y}^c), \dots] \text{ for each } c \in \mathcal{C} . \quad (4.9)$$

The total DICE score on a whole sample (test) set is then simply the average over all classes:

$$\text{dice} = \frac{\sum \overline{\text{DICE}}}{N_C} . \quad (4.10)$$

A DICE score of 0 indicates no overlap between $\mathbf{Y}$ and $\mathbf{P}$, while a score of 1 indicates they are identical.

4.3 Training the Ensembles

For the actual training of each model the UNets **Double Convolutions** are done with convolution parameters `kernel_size` of 3, `padding` of 1 and `stride` of 1 and each **Max-Pool** and **Up-Convolution** operation with `kernel_size` 2 and `stride` 2, which are all

rather standard for UNets, as these are the same parameters [4] states.

The patience for the implemented **Early stopping** is 20 epochs and the **adaptive learning rate**'s LR-Scheduler is patience 15 epochs. This, the Loss and DICE metrics are visible in Figure 4.5 which shows the training graphs of the first three models of an identity ensemble and represents all other model training graphs which were quite similar in the experiments.

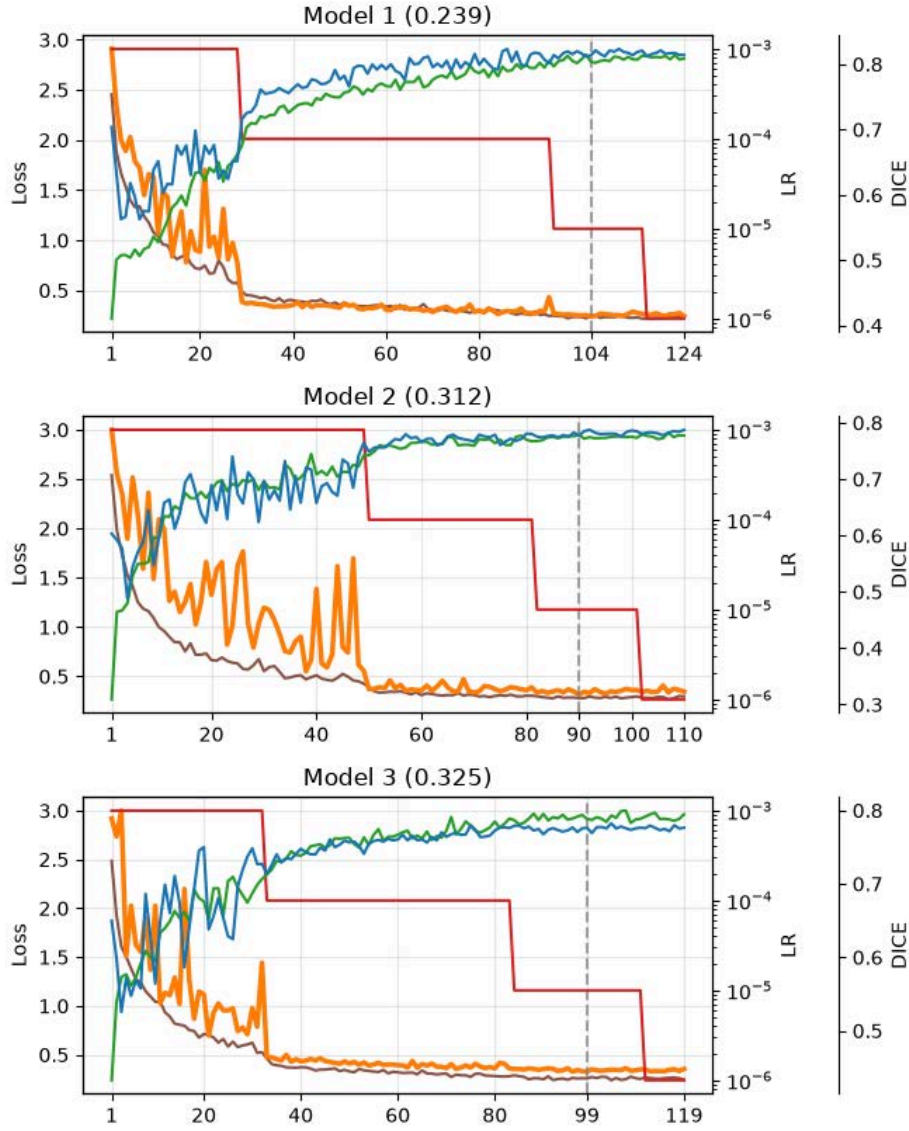


Figure 4.5: Train graphs of three identity modulation models, showing the models metrics over max. 124 epochs: Validation Loss (orange), Training Loss (brown), Learning Rate (red), Mean Validation DICE (blue), Mean Training DICE (green) and Best Validation Loss (grey dashed).

During training in Figure 4.5 the losses and mean DICE scores behave as expected, with decreasing losses where training loss is always slightly lower and the curve smoother since it changes less rapidly per epoch. The validation losses minimum is 0.239 for the best model, shown in bold font. Mean DICE Scores rise towards 1, reaching ~ 0.8 with the

training DICE always being slightly higher than the test DICE. The Adaptive learning rate (LR), initially at 10^{-3} decreases for all models after 45 epochs at latest step by step reaching a maximum final number of 10^{-6} . Looking at all three plots from an ensemble without modulation illustrates how the search for an optimum in the loss function depends heavily on the random initial parameters.

4.4 Results on DFC18

Model and Ensemble result accuracies do vary per training run, so to compare channel modulation strategies themselves in the experiments, not just the effect of random start model params, each ensemble is trained multiple times with the same parameters and compared in an accumulated graph like Figure 4.6. It plots only the Average Accuracy (AA) metric since it is the most commonly used metric stated across other works to summarize a prediction model's performance and compare between approaches.

4.4.1 Baseline

In the modulation system, to train without a modulation means using the **Identity modulation** that leaves data unchanged. The Figure 4.6 plots three ensembles in the left graph, each with the same parameter $N_E = 7$ and two ensembles on the right, both with $N_E = 10$. It shows how individual models' performance varies and that the ensemble's performance is more consistent than model performance across multiple runs without modulation. The influence of parameter N_E being 7 or 10 on the AA is not decisive enough to make a statement about it. Averaging the results, the baseline to reach for the other modulations is an **AA of 54.51%**.

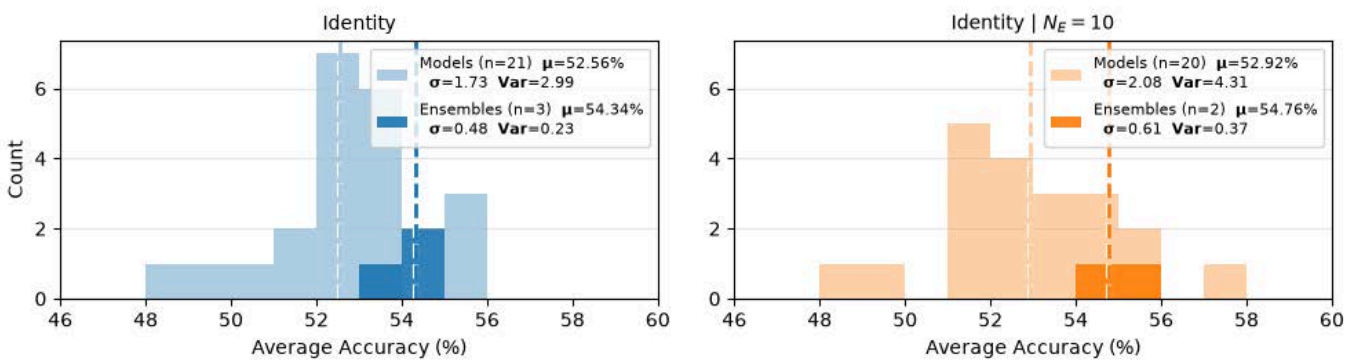


Figure 4.6: Two **Identity** Ensemble's results, showing Average Accuracy of each ensemble's model and each total ensemble

#	Class Name \ Model Num.	#7	#5	#1	#3	#4	#2	#6	E
1	Healthy grass	99.31	99.15	99.05	98.99	98.67	99.15	98.66	99.21
2	Stressed grass	71.94	71.96	67.98	76.46	72.08	74.62	79.81	75.21
3	Artificial turf	0.00	18.11	6.94	25.15	6.20	4.95	4.97	4.79
4	Evergreen trees	97.76	98.50	98.05	97.40	98.41	97.72	98.01	98.53
5	Deciduous trees	75.81	73.98	71.22	76.34	73.67	67.71	72.03	76.29
6	Bare earth	62.45	56.73	48.57	61.12	44.52	43.08	34.70	59.30
7	Water	0.00	2.70	3.69	14.31	7.19	6.39	23.22	7.19
8	Residential buildings	48.68	42.02	47.33	51.71	37.72	36.88	35.81	44.66
9	Non-residential buildings	87.93	92.29	91.58	91.38	91.21	87.88	90.50	92.72
10	Roads	74.72	69.50	76.00	75.00	65.34	65.61	64.45	78.55
11	Sidewalks	64.82	68.11	65.03	65.62	68.85	66.73	66.04	70.79
12	Crosswalks	44.36	41.47	40.78	38.12	38.58	45.90	49.11	45.36
13	Major thoroughfares	34.27	27.83	28.26	28.96	25.30	33.25	27.58	27.65
14	Highways	23.90	12.69	24.13	18.45	23.45	16.31	9.75	19.79
15	Railways	5.78	5.12	0.00	5.61	1.66	6.86	6.63	5.97
16	Paved parking lots	91.18	71.35	86.11	70.53	86.64	64.90	84.71	85.02
17	Unpaved parking lots	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
18	Cars	69.46	56.42	62.70	59.40	69.08	61.63	62.41	66.62
19	Trains	92.51	83.85	57.93	60.75	86.55	85.36	48.05	81.21
20	Stadium seats	56.23	69.56	79.35	49.29	53.69	55.71	49.85	58.62
OA (%)		62.41	60.09	59.80	59.78	59.33	57.27	55.39	61.77
AA (%)		55.05	53.07	52.73	53.23	52.44	51.03	50.31	54.87
Kappa		0.60	0.58	0.57	0.57	0.57	0.55	0.53	0.59
Avg DICE		0.82	0.82	0.82	0.83	0.81	0.81	0.81	0.84

Table 4.1: An *Identity Ensemble*'s results, showing performance on each DFC18 class

In Table 4.1 the columns are the model's and the total ensemble's performance on each class as well as Overall- and Average Accuracy, Kappa and average DICE-Score, with the highest value per row in **bold**. The model columns #1 - #7 are sorted by OA with the highest on the left. The **top model** (#7) has 55.05% AA, the **Ensemble (E)** 54.87% and the other models lower values.

This suggests that just the ensemble technique of majority voting improves Average Accuracy in comparison to just using one model. Which is supported by Figure 4.6 above showing identity ensembles total AA being higher than most singular identity model's AAs.

4.4.2 Using Modulated Ensembles

Figure 4.7 is a comparison over all modulations with each having their default parameters set as in Section 3.1. It shows how **Powerset of all modalities** and **PCA** yield a performance higher than without modulation (AA 54.87%), **Random linear combination** a lower accuracy and the others a similar accuracy.

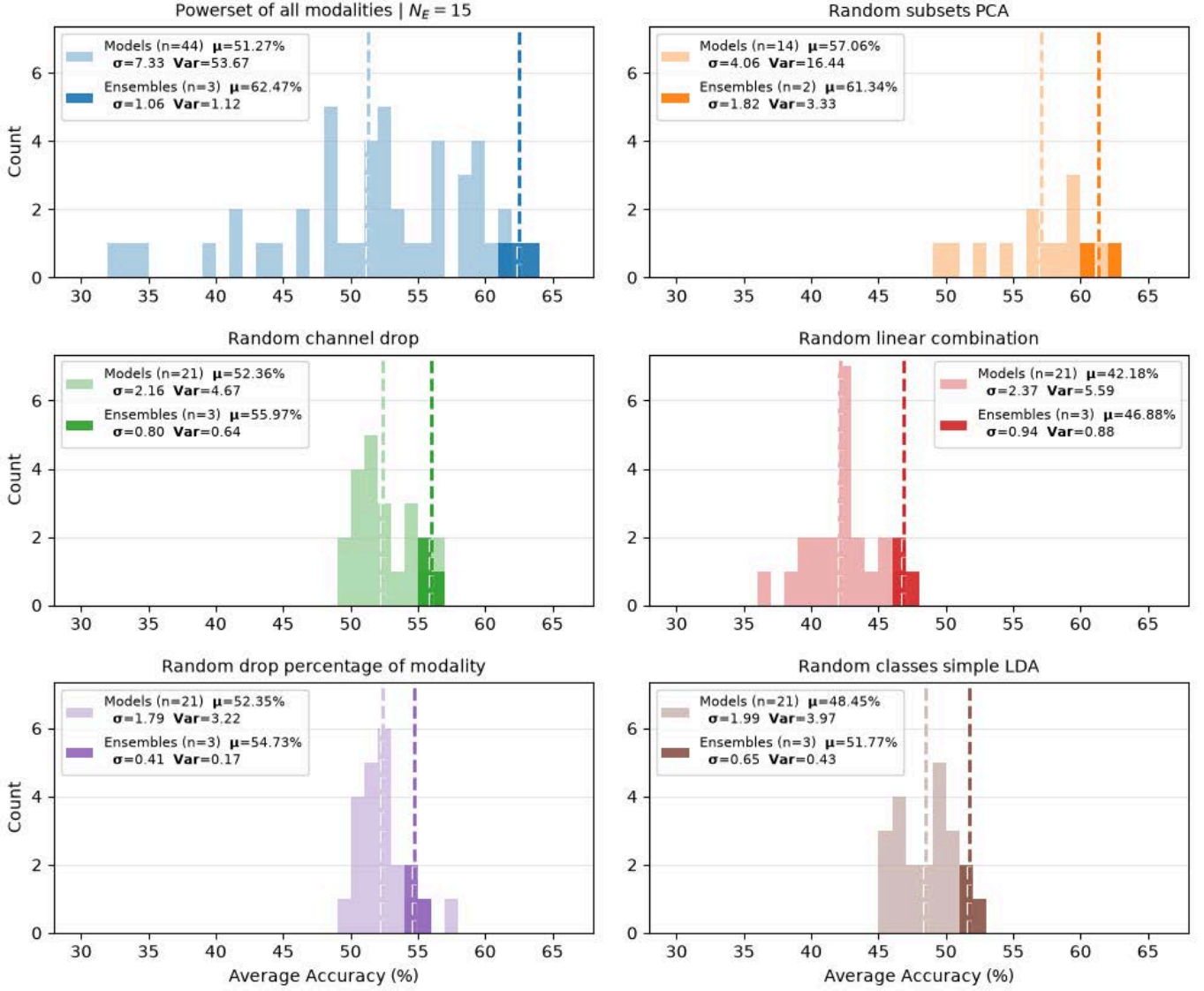


Figure 4.7: Distribution of Average Accuracy across ensemble runs & their models

A few modulations results are covered in more detail below, also discussing the impact of varying ensemble modulation parameters.

4.4.3 PCA

Parameter experimentation is conducted here to understand the influence of the ensemble model count N_E , previous modulations and the PCA's hyperparameters: components K and subsets sample fraction p in Figure 4.8. Their values are omitted when $K = 55$ (all DFC18 channels) or $p = 0.75$. The exact values tested were empirically chosen and “prev: powerset of all modalities” indicates that **Powerset of all modalities** was chained before

PCA.

In all cases the ensemble have higher results to no modulation (AA 54.87%), with the chained modulation reaching a record of 65.31% AA. The worst performance is when $K = 10$: too few channels, even of the most important principal components, to describe the data fully. To clearly declare if $p = 0.75$ or $p = 0.5$ is superior more experiments need to be conducted.

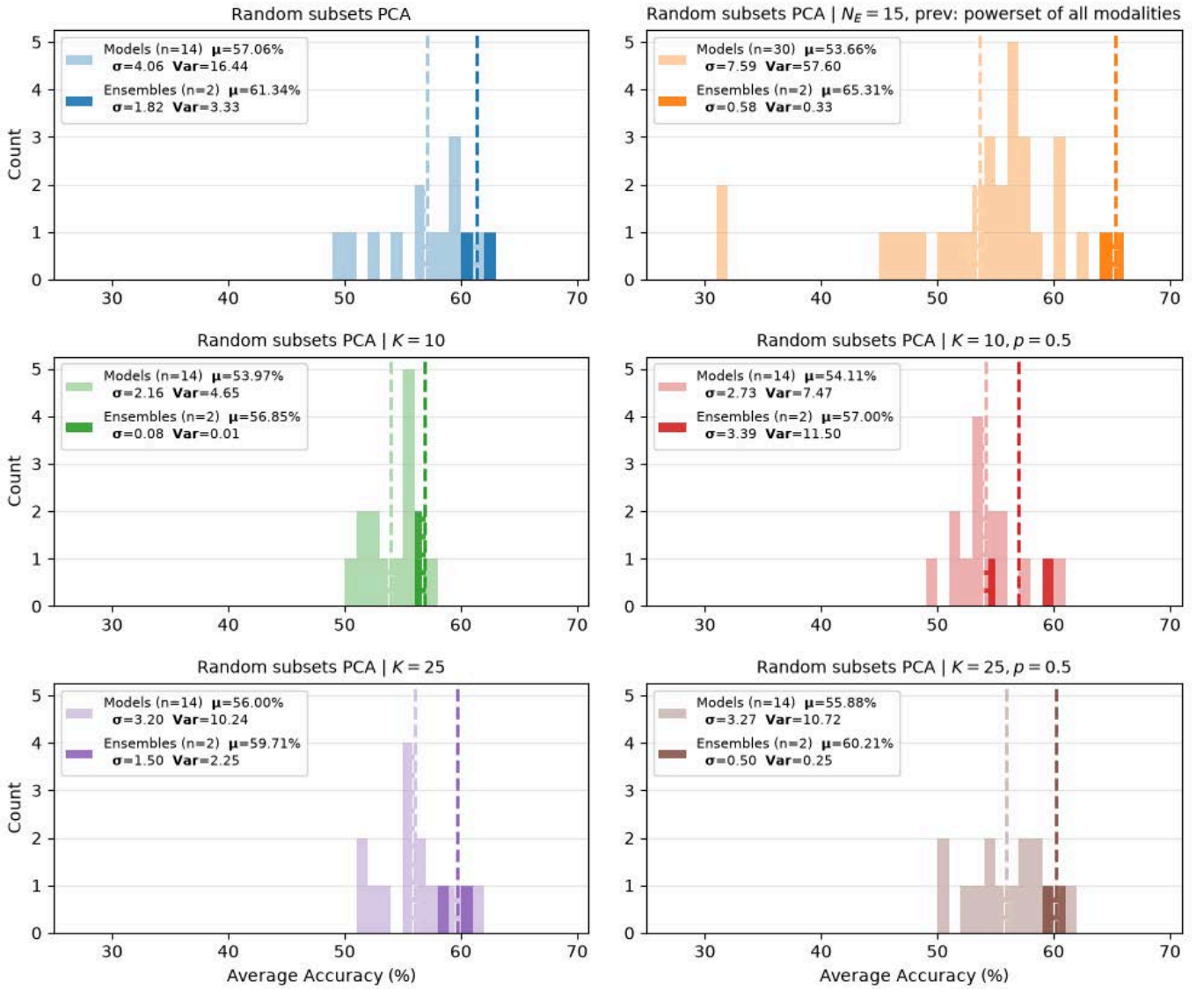


Figure 4.8: Distribution of Average Accuracy across *PCA* ensemble runs & their models with differing hyperparameters and a chained modulation previous to *PCA*.

4.4.4 Powerset of all modalities (chained with PCA)

This modulation is special since it is clearly defined which modalities are fed into which model, which is visualized for an ensemble together with its results in Table 4.2. The header row states the modalities that were fed into the model of that column.

#	Class Name \ Model Num.	RGB	HSI	LIDAR INT	LIDAR DSM	RGB, HSI	RGB, LIDAR INT	RGB, LIDAR DSM	HSI, LIDAR INT	HSI, LIDAR DSM	LIDAR INT, LIDAR DSM	RGB, HSI, LIDAR INT	RGB, HSI, LIDAR DSM	HSI, LIDAR INT, LIDAR DSM	RGB, HSI, LIDAR INT, LIDAR DSM	E	
1	Healthy grass	96.70	92.34	96.96	0.07	97.68	97.52	97.03	97.98	99.27	96.86	98.50	99.05	97.27	99.00	98.44	99.21
2	Stressed grass	55.60	71.47	46.05	17.50	81.18	54.65	58.26	67.51	70.31	45.36	74.75	74.06	57.67	58.61	66.93	70.39
3	Artificial turf	31.77	0.20	17.70	0.00	18.93	79.11	0.00	17.65	78.93	4.77	84.64	76.71	6.62	65.62	84.06	76.81
4	Evergreen trees	96.75	85.08	98.15	60.34	94.85	98.94	98.76	94.31	94.96	97.15	89.44	95.55	97.68	97.07	96.09	98.76
5	Deciduous trees	42.40	51.31	77.35	79.16	55.15	80.96	74.62	68.99	58.15	78.42	74.19	58.34	79.68	53.82	69.47	81.38
6	Bare earth	16.09	16.28	82.37	0.00	35.54	68.56	6.66	37.51	37.18	18.45	42.19	34.49	20.62	0.89	20.13	35.90
7	Water	10.01	63.45	7.13	0.00	10.50	8.17	9.46	8.60	86.98	4.18	26.72	1.97	9.64	97.73	30.16	43.43
8	Residential buildings	78.90	52.82	79.02	77.10	29.16	83.38	77.81	63.13	71.72	84.18	30.78	30.63	85.56	61.65	54.73	82.01
9	Non-residential buildings	87.33	79.68	76.29	77.96	81.66	87.53	87.57	82.96	87.70	90.16	81.82	89.97	91.29	90.17	90.45	94.72
10	Roads	68.84	66.85	59.85	30.07	83.45	62.41	65.16	63.44	79.56	76.29	83.38	78.86	63.65	85.85	80.82	89.35
11	Sidewalks	66.88	49.79	64.24	36.16	55.01	67.80	73.74	54.60	58.88	66.41	58.40	63.89	75.04	52.09	64.43	72.11
12	Crosswalks	68.57	16.19	69.50	0.00	13.95	74.46	55.72	33.77	20.07	68.83	43.61	15.47	67.22	35.29	39.24	41.73
13	Major thoroughfares	23.89	56.77	41.02	10.50	41.95	31.60	30.17	59.70	45.06	23.70	41.43	28.98	29.42	41.81	40.77	38.76
14	Highways	19.11	24.73	18.16	15.70	17.10	18.92	19.27	16.27	11.63	23.36	11.97	11.24	24.49	14.58	15.37	18.39
15	Railways	94.88	7.47	91.95	1.52	7.41	98.13	96.79	7.45	7.33	21.63	7.43	4.96	98.63	7.43	7.48	55.54
16	Paved parking lots	72.90	32.97	78.56	77.57	15.23	83.06	83.81	36.47	31.02	79.85	54.57	49.60	85.76	47.10	73.99	72.78
17	Unpaved parking lots	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
18	Cars	54.43	25.52	63.78	75.88	34.49	62.84	60.97	27.79	43.34	66.16	55.26	46.96	66.43	48.85	62.33	65.08
19	Trains	30.07	29.91	90.76	41.74	82.62	98.04	58.31	35.76	71.14	91.34	83.11	61.31	62.19	52.24	83.24	82.51
20	Stadium seats	89.92	92.79	96.10	21.45	95.05	32.06	32.97	96.25	81.93	83.60	98.03	98.57	50.19	85.02	78.49	95.49
	OA (%)	58.76	49.29	67.74	36.40	54.32	69.26	58.24	54.69	60.73	61.85	63.27	58.87	62.49	57.12	64.22	71.39
	AA (%)	55.25	45.78	62.75	31.14	47.55	64.41	54.35	48.51	56.76	56.03	57.01	51.03	58.45	54.74	57.83	65.72
	Kappa	0.56	0.46	0.66	0.33	0.52	0.67	0.56	0.52	0.58	0.60	0.61	0.56	0.60	0.55	0.62	0.70
	Avg DICE	0.83	0.80	0.81	0.76	0.82	0.83	0.82	0.82	0.83	0.83	0.83	0.83	0.83	0.83	0.84	0.88

Table 4.2: Results of *Powerset of all modalities* chained before *PCA*: 15 models on DFC18 with each model column stating the combination of modalities it received.

The results of each **Powerset of all modalities** ensemble are very similar for each run: The model that just receives LiDAR-Intensities is the best single model, with only the ensemble prediction performing better.

To improve the performance even further **Powerset of all modalities** and **PCA** were chained as one modulation giving the ensemble results in Table 4.2 of an OA of 71.39% and all other metrics as the highest as well: AA = 65.72%, Kappa = 0.70 and average DICE-

Score = 0.88. There then the best single model becomes the RGB + LiDAR-Intensities model for both OA and AA.

4.4.5 Random channel drop

Experimenting in detail with parameters of this modulation: The drop probability p together with a varying model count N_E in the ensemble in Figure 4.9. Their values are omitted in the plot when $N_E = 7$ or $p = 0.3$.

For all value combinations the resulting ensemble AA is at least the **Identity**'s score (AA 54.87%) except for drop probability $p = 0.8$ where the ensembles perform poorly. A higher N_E tends to increase the stability of the ensemble's AA with the $N_E = 15$ ensembles having a similar or higher accuracy than their $N_E = 7$ counterpart with the same p , though more experiments are needed to precisely verify this. The same applies to deciding which p out of 0.1, 0.3 and 0.5 is best for a high AA.

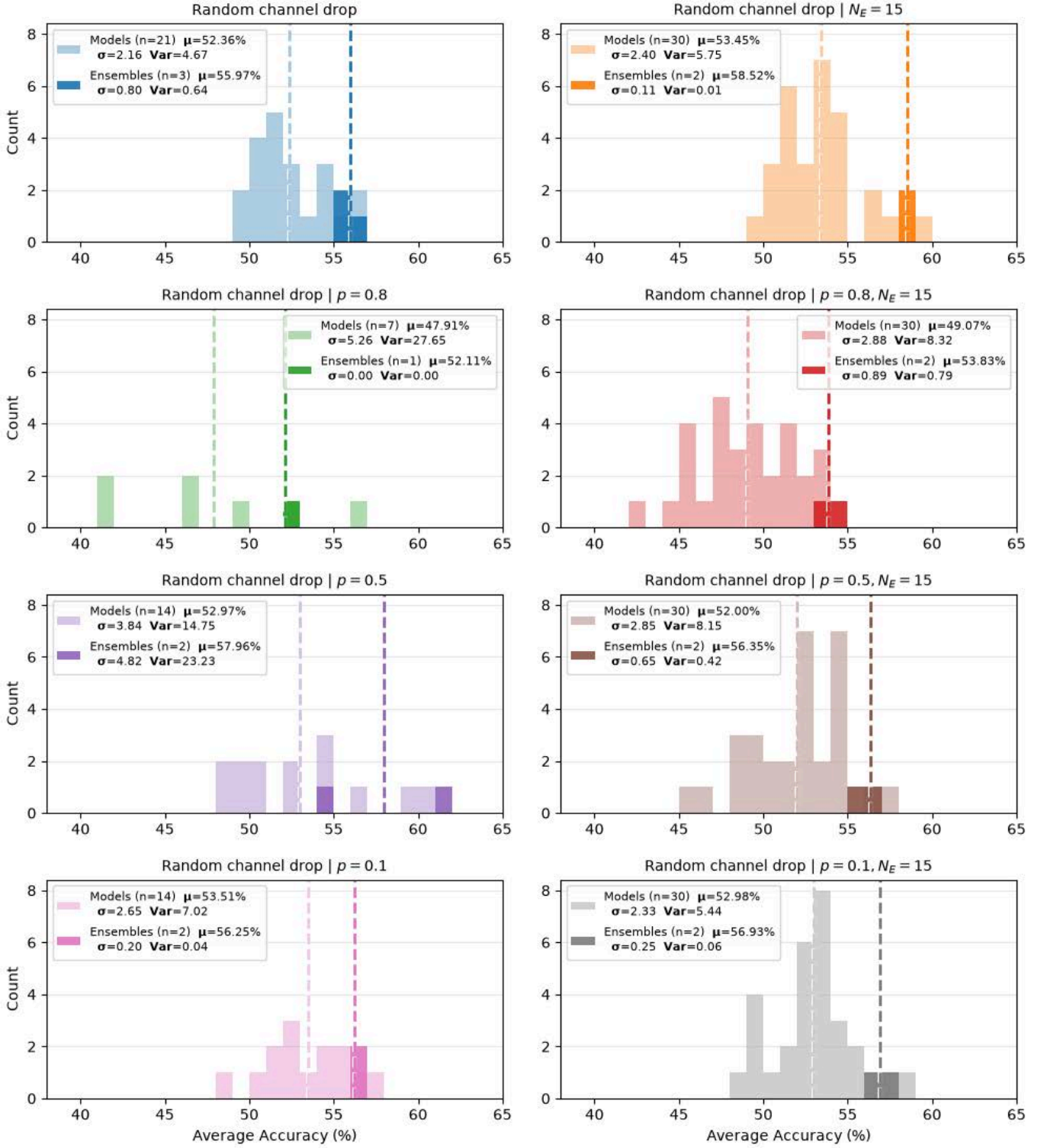


Figure 4.9: Distribution of Average Accuracy across *Random channel drop* ensemble runs & their models with differing hyperparameters

4.4.6 Random Classes Simple LDA

A parameter experimentation with the count of input channels of the model K can be seen in Figure 4.10. There does not seem to be a connection between AA and K as all ensembles independent of K have an AA of around 52%, lower than without modulation (AA 54.87%).

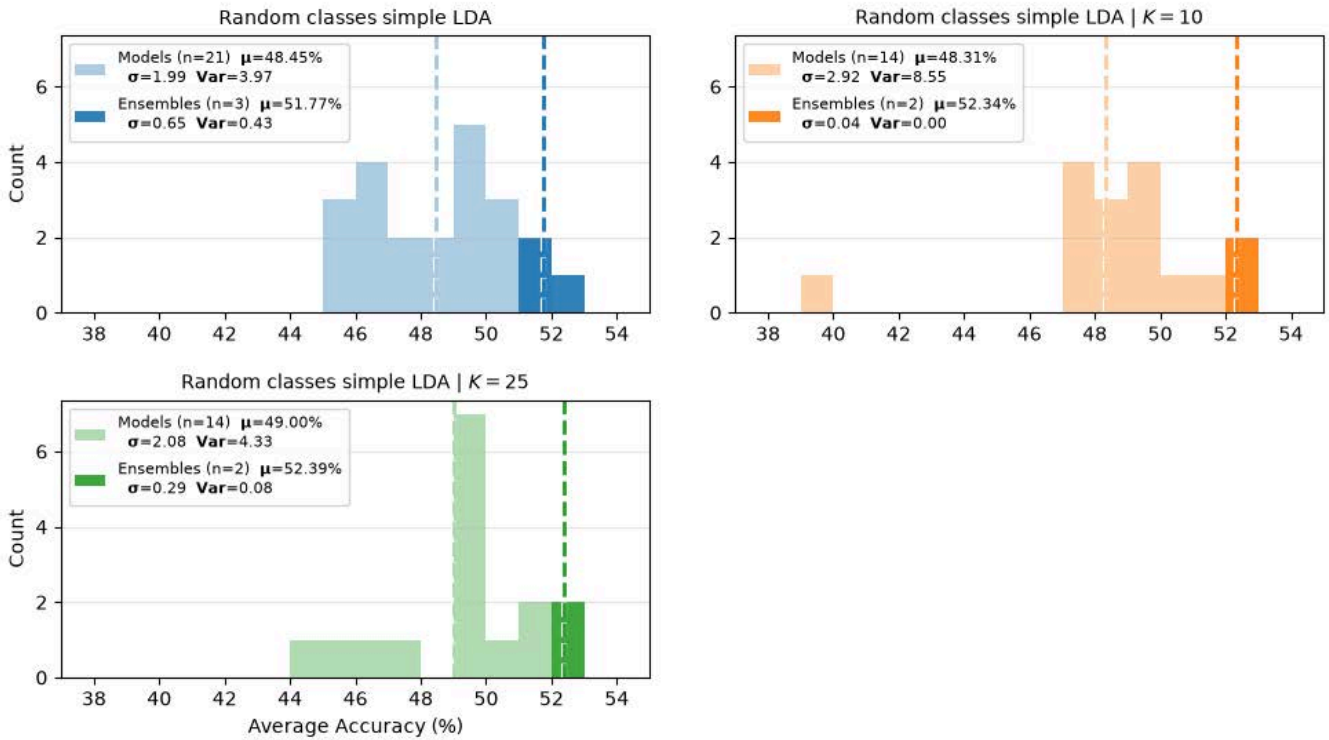
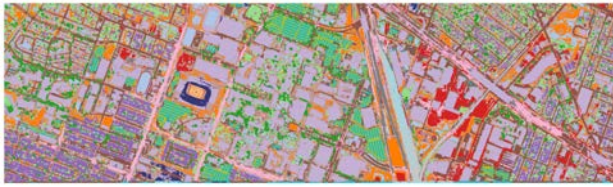


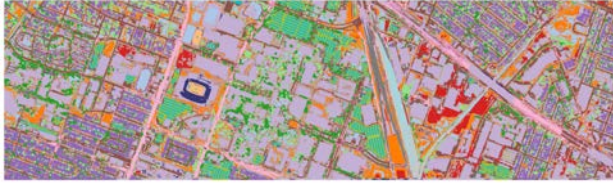
Figure 4.10: Distribution of Average Accuracy across *Random Classes Simple LDA* ensemble runs & their models with differing hyperparameter

Ensemble prediction on the full DFC18 dataset area

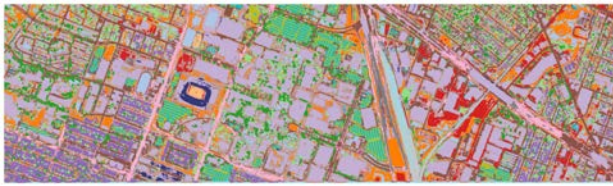
The full area images in Figure 4.12 and Figure 4.13 depict per modulation the ensemble predictions and its entropy (certainty) per pixel of the test and trainset. When looking closely, a checkerboard-like pattern is visible in the label maps as well as some entropy maps, which appears since during dataset tiling, there is no overlapping stride, so each tile's edge behaves slightly differently from its center.



Identity



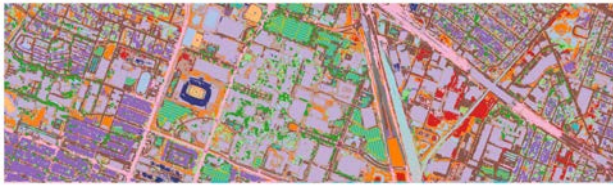
Powerset of All Modalities



Random Channel Drop



Random Drop Percentage of Modality



PCA

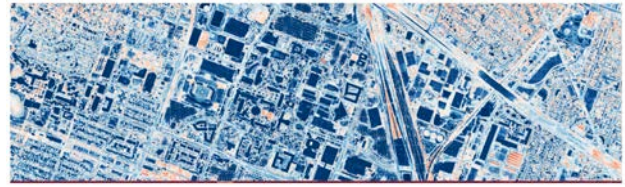


Random Classes Simple LDA



Random Linear Combination

Figure 4.12: DFC18 Label maps predicted by ensembles with the same base learner but varying modulations. Note that images contain both train and test regions.



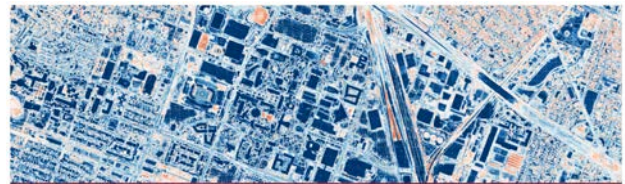
Identity



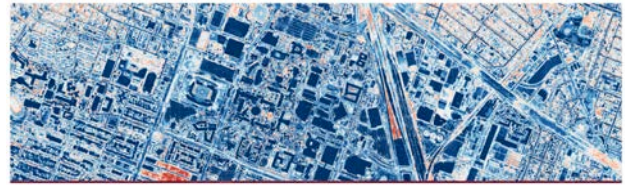
Powerset of All Modalities



Random Channel Drop



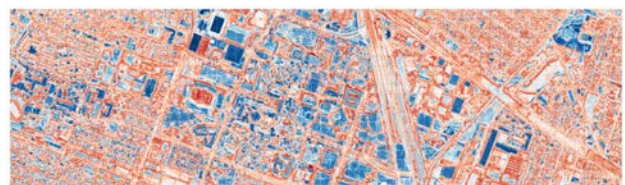
Random Drop Percentage of Modality



PCA



Random Classes Simple LDA



Random Linear Combination

Figure 4.13: Entropy of the predicted class by ensembles with the same base learner but varying modulations. Colormap is the same as used for the modulations: Gradient from blue (-1), through white (0) to red (+1)

4.5 Results on MDAS

4.5.1 Baseline

Table 4.3 shows the same as for DFC18 in Table 4.1: Just the ensemble technique of majority voting improves Average Accuracy in comparison to just using one model with the best model having an AA of 85.01% and the ensemble 85.63%.

#	Class Name \ Model Num.	#5	#4	#1	#7	#2	#3	#6	E
1	Forest	87.83	90.00	88.55	87.21	86.05	88.47	86.24	88.60
2	Residential	90.68	89.01	89.50	90.14	91.25	86.77	88.13	90.43
3	Commercial	90.24	90.81	91.14	88.26	86.78	91.71	90.33	90.77
4	Low Plant	88.25	88.41	88.19	88.06	87.56	89.02	87.87	88.94
5	Allotments	84.64	84.34	83.38	86.88	82.09	89.73	85.97	87.28
6	Recreation Ground	53.69	63.23	65.61	65.62	58.31	61.54	66.59	64.60
7	Water	88.23	87.99	88.74	86.06	89.13	86.61	88.55	88.78
	OA (%)	88.85	88.68	88.68	88.43	88.23	88.09	87.79	89.34
	AA (%)	83.37	84.83	85.01	84.60	83.02	84.83	84.81	85.63
	Kappa	0.85	0.84	0.84	0.84	0.84	0.84	0.83	0.85
	Avg DICE	0.81	0.83	0.83	0.81	0.81	0.83	0.81	0.84

Table 4.3: An *Identity* Ensemble's results, showing performance on each MDAS class

4.5.2 Using Modulated Ensembles

The aggregated performances of all modulations can be seen in Figure 4.14, where the mean AA lies above no-modulation (AA 85.46%) for **Powerset of all modalities** (AA 86.57%), **Random subsets PCA** (AA 87.3%) and the chaining of these two while all others report the same AA as no-modulation or lower. The highest AA of Ensembles with is the **chained Powerset of all modalities after PCA** an Accuracy of 87.90%.

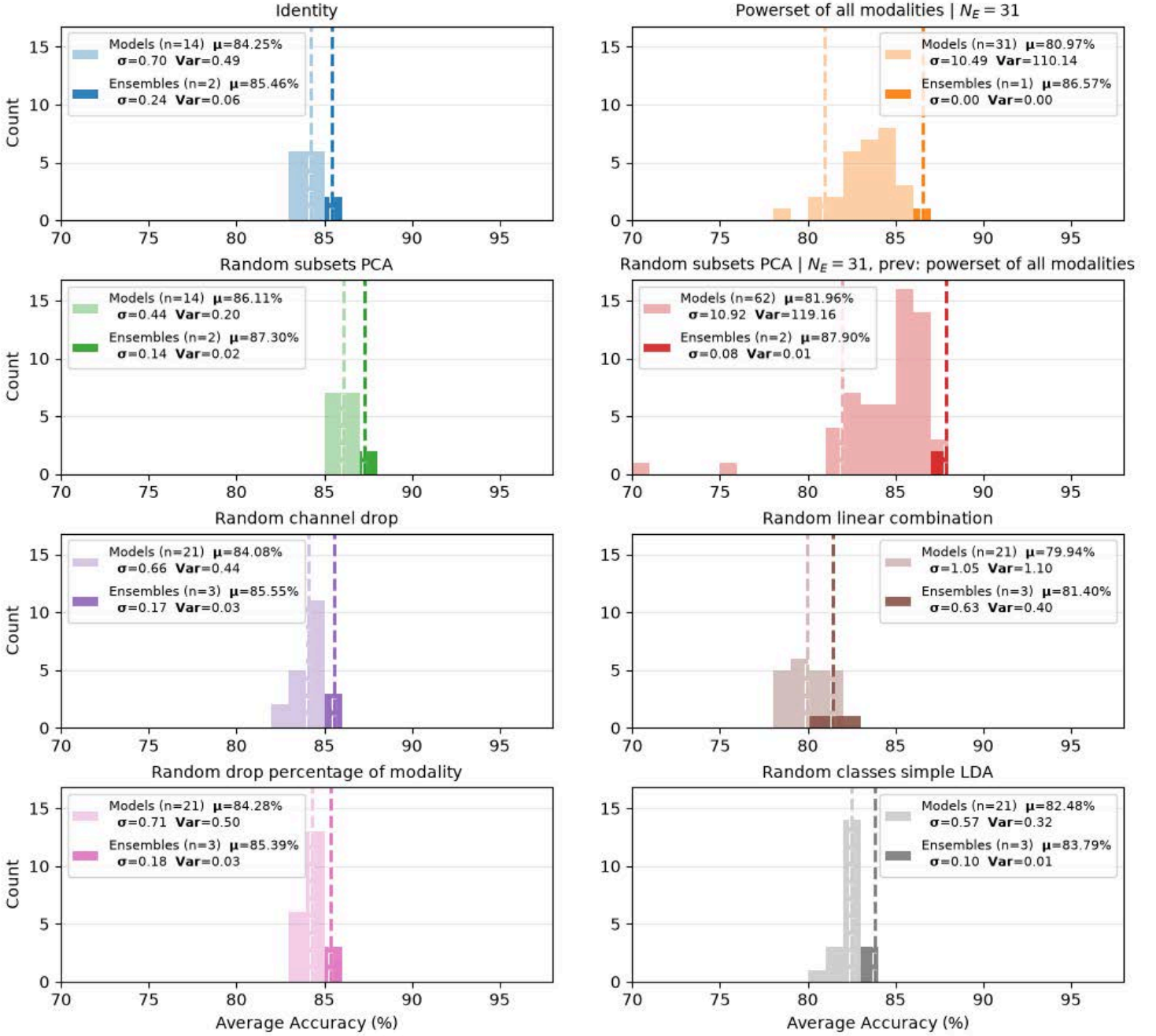


Figure 4.14: Distribution of Average Accuracy across ensemble runs & their models and *each their top performing parameters*

4.5.3 Powerset of all modalities

MDAS with its 5 modalities, so one more modality than DFC18, has a significantly larger **Powerset of all modalities** ensemble size $N_E = \Pi(5) = 31$ (calculated per Section 3.1.4)

as seen in Table 4.4. The model that just receives RGB and DSM (OA 89.29) is the best single model, with only the ensemble prediction performing better (OA 90.51%).

#	Class Name \ Model Num.	RGB	HSI	MSI	SAR	DSM	RGB, HSI	RGB, MSI	RGB, SAR	RGB, DSM	HSI, MSI	HSI, SAR	HSI, DSM	MSI, SAR	MSI, DSM
1	Forest	86.50	87.68	87.38	94.97	84.36	85.09	86.28	87.15	89.37	83.39	86.74	88.63	83.73	86.28
2	Residential	88.13	90.79	90.56	78.10	88.89	91.24	89.99	91.41	90.47	89.28	89.28	89.69	88.04	87.72
3	Commercial	89.69	85.44	87.06	16.01	86.96	87.36	88.87	88.29	90.02	88.41	87.01	89.37	87.43	88.22
4	Low Plant	85.77	88.28	85.38	0.00	83.82	86.90	90.40	88.73	89.55	87.61	87.85	89.43	83.54	87.78
5	Allotments	84.03	82.59	78.59	0.24	81.53	86.08	85.58	84.07	88.82	86.06	84.29	86.60	77.69	79.03
6	Recreation Ground	61.21	55.89	52.96	0.00	48.21	58.13	61.84	54.93	50.55	63.73	62.04	59.64	59.88	68.03
7	Water	76.39	86.46	87.16	0.00	73.69	85.08	86.77	81.23	81.03	87.15	86.31	86.96	85.62	87.64
	OA (%)	86.65	88.27	87.20	41.18	85.44	87.97	89.10	88.85	89.29	87.53	87.75	88.98	85.29	87.20
	AA (%)	81.68	82.45	81.30	27.05	78.21	82.84	84.25	82.26	82.83	83.66	83.36	84.33	80.85	83.53
	Kappa	0.82	0.84	0.82	0.25	0.80	0.83	0.85	0.85	0.85	0.83	0.83	0.85	0.80	0.82
	Avg DICE	0.77	0.81	0.80	0.77	0.81	0.81	0.81	0.81	0.82	0.80	0.81	0.83	0.76	0.80

SAR, DSM	RGB, HSI, MSI	RGB, HSI, SAR	RGB, HSI, DSM	RGB, MSI, SAR	RGB, MSI, DSM	RGB, SAR, DSM	HSI, MSI, SAR	HSI, MSI, DSM	HSI, SAR, DSM	MSI, SAR, DSM	RGB, HSI, MSI, SAR	RGB, HSI, MSI, DSM	RGB, HSI, SAR, DSM	RGB, MSI, SAR, DSM	HSI, MSI, SAR, DSM	RGB, HSI, MSI, SAR, DSM	E
83.47	86.54	85.88	87.54	84.79	87.40	86.49	85.20	87.60	85.84	83.55	87.84	89.09	89.58	85.76	87.85	87.71	89.36
83.19	87.93	89.19	89.65	89.49	89.20	88.58	89.36	90.77	86.59	88.65	89.95	88.79	89.91	89.58	89.16	88.65	92.16
77.59	87.31	87.33	89.02	87.97	90.02	90.88	88.93	88.16	89.35	84.37	89.61	90.39	88.78	89.52	89.50	86.59	91.09
79.26	86.82	89.79	87.66	90.44	89.73	88.96	88.24	88.86	88.81	81.13	87.57	88.67	88.59	90.56	89.15	87.43	90.01
55.41	85.04	86.51	85.32	87.24	84.16	87.15	83.98	86.20	83.71	77.60	84.21	85.16	85.06	83.18	85.46	84.55	87.64
45.56	64.98	63.55	61.65	70.03	65.09	55.32	66.77	68.31	66.17	61.60	60.94	60.48	49.04	71.24	60.10	65.34	66.96
50.47	86.96	87.28	89.08	86.58	85.46	83.48	86.91	86.48	87.14	86.77	87.80	88.91	89.42	86.60	88.73	87.64	88.74
79.43	86.97	88.43	88.21	88.79	88.81	88.23	88.01	89.00	87.30	84.34	88.33	88.55	88.69	89.07	88.62	87.50	90.51
67.85	83.66	84.22	84.27	85.22	84.44	82.98	84.20	85.20	83.94	80.52	83.99	84.50	82.91	85.20	84.28	83.99	86.57
0.72	0.82	0.84	0.84	0.85	0.85	0.84	0.84	0.85	0.83	0.79	0.84	0.84	0.84	0.85	0.84	0.83	0.87
0.78	0.79	0.82	0.82	0.80	0.81	0.80	0.81	0.82	0.80	0.78	0.82	0.82	0.82	0.81	0.82	0.81	0.86

Table 4.4: Results of *Powerset of all modalities* 31 models on MDAS with each model column stating the combination of modalities it received.

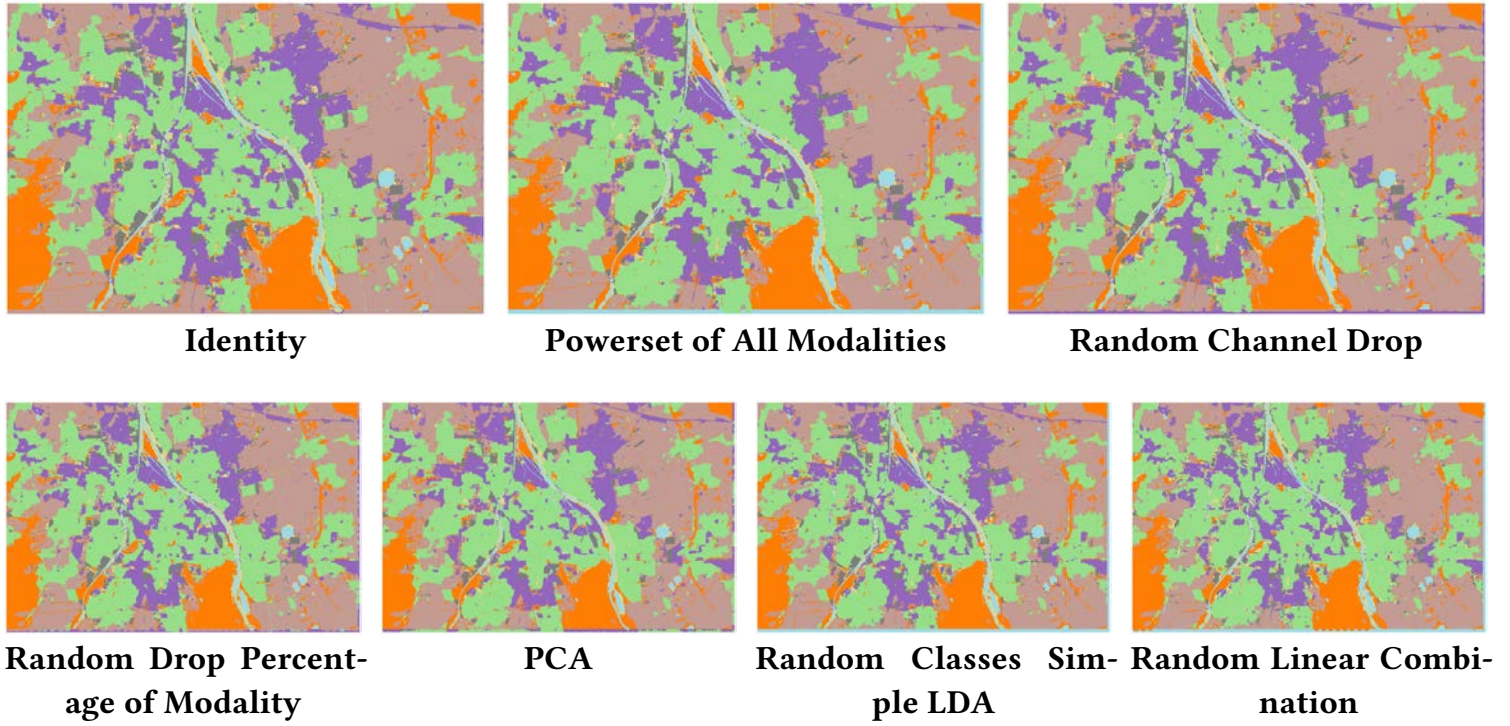


Figure 4.16: MDAS Label maps predicted by ensembles with the same base learner but varying modulations. Note that images contain both train and test regions.

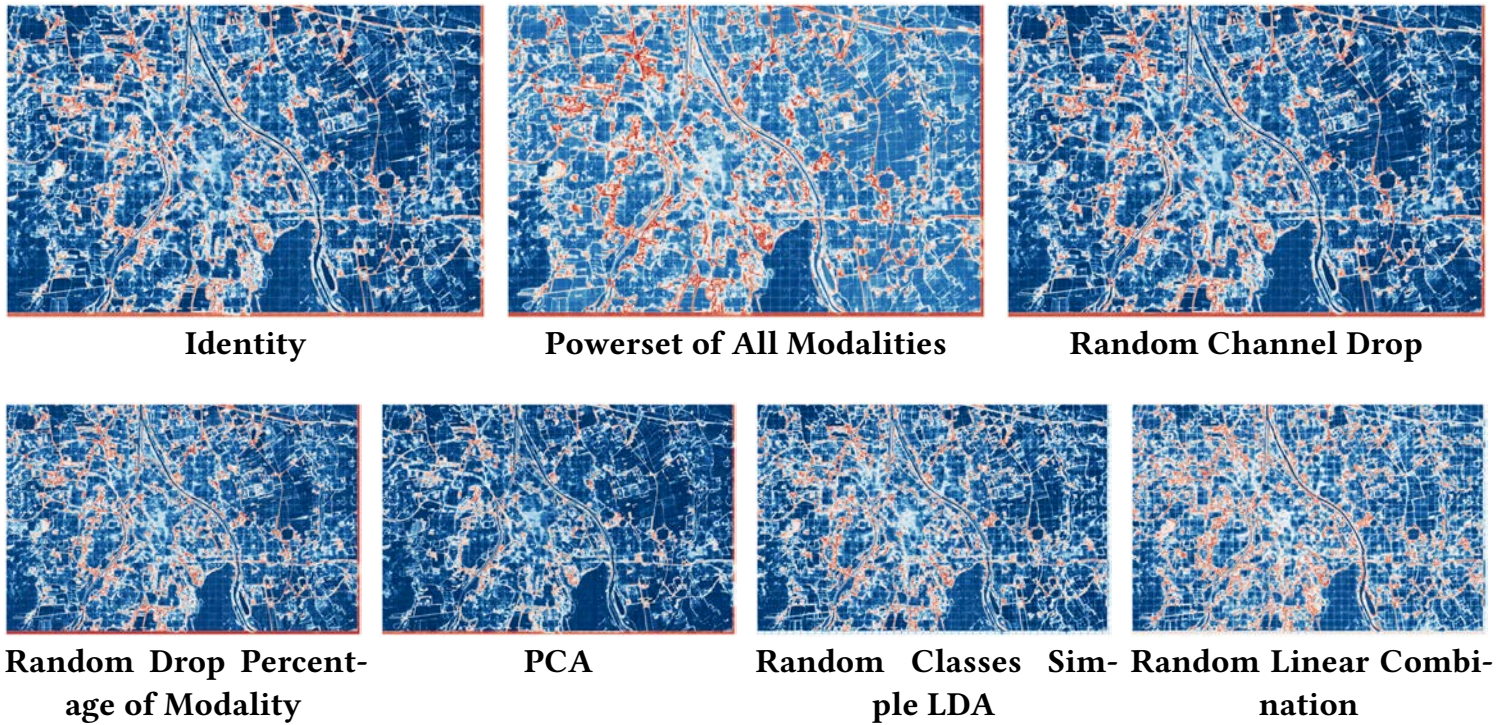


Figure 4.17: Entropy of the predicted class by ensembles with the same base learner but varying modulations. Colormap is the same as used for the modulations: Gradient from blue (-1), through white (0) to red (+1)

Ensemble prediction on the full MDAS dataset area

Figure 4.16 and Figure 4.17 show the prediction on the full MDAS dataset area and the entropy per pixel. Again a checkerboard-like pattern can be perceived in the entropy visualization.

4.6 Conclusions

The **Identity** ensembles show that just the ensemble technique of majority voting improves Average Accuracy in comparison to just using one model, since **Identity** ensembles total AA are higher than most singular identity model's AAs. So the ensemble technique helps cancel out the influence of random initial model parameters on the search for an optimum in the loss function.

Powerset of all modalities and **PCA** work well, improving all metrics relative to identity ensembles, with an average AA on DFC18 of 62.47 61.34% respectively. They all diversify well because they hide data from each member at some point, making each strong on a subset of channel or samples and giving a strong average ensemble prediction. As shown here with **Powerset of all modalities**: The LiDAR intensities are the single best datasource for a simple predictor on DFC18. This is the same result [8] arrived at.

The grid-like parameter search on **Random channel drop** ensembles doesn't result in a decisive winning parameter combination of N_E and p but that dropping 80% of channels per member is too much and is better between 10% and 50%. Also a higher N_E tends to increase the stability of the ensemble's AA. Similarly, the parameter exploration on **PCA** concludes that a low principal components count K of 10 has a slightly negative effect on performance, while K of 25 or above all give the same baseline performance. There does not seem to be a stark difference in if the sample percentage p is 0.5 or 0.75.

Three modulations perform poorly in all experiment runs as seen in Figure 4.7 and Figure 4.14: **Random linear combination** does not diversify the ensemble and instead make each member a poor predictor since the original data gets combined with strong random noise. The **Random drop percentage of modality** and **Random channel drop** neither meaningfully diversify ensembles and lead to no-modulation-like AA results.

Picking a low modality, so therefore an often high information density modality and removing a fixed 30% of channels loses information more often than it diversifies. A little more hope was set into **Random classes simple LDA** and into trying to find a better parameter K for it but here does not seem to be a connection between AA and K . This modulation has multiple problems, among those are: That each of the K directions comes from a single random pixel pair, not an average over many samples or true class means, so there is a lot of noise in this selection. Another suboptimal design choice is that class pairs and pixels are drawn independently, so two of the K columns risk landing on similar or even identical class pairs, creating redundant projections.

Interestingly the entropy of the well performing **Powerset of all modalities** ensembles prediction is high, while it is low for all others (except **Random linear combination** where it's expected) as seen in Figure 4.13. The reason for this might be the very high diversity with large N_E of **Powerset of all modalities** ensembles.

This papers best experiments results on DFC18 come from chaining **Powerset of all modalities** and **PCA** (Table 4.2): An average AA of 65.31%, OA of 71.39% and kappa of 0.70 and average DICE-Score of 0.88.

Comparing with other works The results shown above in Chapter 2 on DFC18 from Random Forests [8] have a maximum OA of 67.54%, AA of 62.92% and of kappa 0.66, while the results from [5] have a maximum OA of 63.28%, AA of 56.5% and kappa of 0.61 for the neural network models without heavy pre- or postprocessing. Therefore it is clear that the modulations approach does improve performance on DFC18, compared to both other approaches.

Since for MDAS a custom dataset was created, numerically comparing to other works is not feasible.

On the datasets When looking at the accuracies per class on DFC18, some classes are only poorly learned by the vast majority of models: “*Unpaved parking lots*”, “*Water*”, “*Artificial Turf*” and “*Railways*”. This can be explained with a low sample count for these classes, eg only 2 tiles of the trainset having class “*Unpaved parking lots*” which can easily end up in the validation set so the model never sees it during training. There are a few models that do manage to learn the latter three classes but that must just be a model where all of the few tiles with these classes were shuffled into the trainset.

The performances on MDAS are much higher (up to 86% Average Accuracy) than on DFC18 simply because the latter dataset's training and testing areas are spatially separated from each other, whereas MDAS train tiles are randomly selected and can lay directly next to test tiles and its reference data is less detailed therefore train and test tiles being more similar more often. This can be the reason for kappa being lower than AA for most MDAS models which is rarely the case for DFC18 models, so the model also does at least partially rely on guessing the most common classes.

All in all the modulations for ensembles approach does work, the modulations just need to be chosen carefully so they do actually improve diversity in the ensemble while keeping individual members strong. With that, choosing a higher ensemble members count has produced more reliable results.

Bibliography

- [1] J. Hu *et al.*, “MDAS: a new multimodal benchmark dataset for remote sensing,” *Earth System Science Data*, vol. 15, no. 1, pp. 113–131, 2023, doi: [10.5194/essd-15-113-2023](https://doi.org/10.5194/essd-15-113-2023).
- [2] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996, doi: [10.1007/BF00058655](https://doi.org/10.1007/BF00058655).
- [3] A. Krogh and J. Vedelsby, “Neural Network Ensembles, Cross Validation, and Active Learning,” in *Advances in Neural Information Processing Systems*, MIT Press, 1995.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Springer International Publishing, 2015, pp. 234–241. doi: [10.1007/978-3-319-24574-4_28](https://doi.org/10.1007/978-3-319-24574-4_28).
- [5] Y. Xu *et al.*, “Advanced Multi-Sensor Optical Remote Sensing for Urban Land Use and Land Cover Classification: Outcome of the 2018 IEEE GRSS Data Fusion Contest,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 6, pp. 1709–1724, 2019, doi: [10.1109/JSTARS.2019.2911113](https://doi.org/10.1109/JSTARS.2019.2911113).
- [6] D. Hong *et al.*, “More Diverse Means Better: Multimodal Deep Learning Meets Remote-Sensing Imagery Classification,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 5, pp. 4340–4354, 2021, doi: [10.1109/TGRS.2020.3016820](https://doi.org/10.1109/TGRS.2020.3016820).
- [7] J. Rodriguez, L. Kuncheva, and C. Alonso, “Rotation Forest: A New Classifier Ensemble Method,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 10, pp. 1619–1630, 2006, doi: [10.1109/TPAMI.2006.211](https://doi.org/10.1109/TPAMI.2006.211).
- [8] R. Hänsch and O. Hellwich, “Fusion of Multispectral LiDAR, Hyperspectral, and RGB Data for Urban Land Cover Classification,” *IEEE Geoscience and Remote Sensing Letters*, vol. 18, no. 2, pp. 366–370, 2021, doi: [10.1109/LGRS.2020.2972955](https://doi.org/10.1109/LGRS.2020.2972955).
- [9] I. Burenko and D. Vetrov, “Beyond Modality Fusion: Deep Ensembles for Multimodal Classification,” 2026, [Online]. Available: <https://arxiv.org/abs/2607.05019>
- [10] F. Imrie, S. Denner, L. S. Brunshwig, K. Maier-Hein, and M. van der Schaar, “Auto-Prognosis-M: Automated Ensemble Multimodal Machine Learning for Healthcare,” 2024, [Online]. Available: <https://arxiv.org/abs/2407.18227>